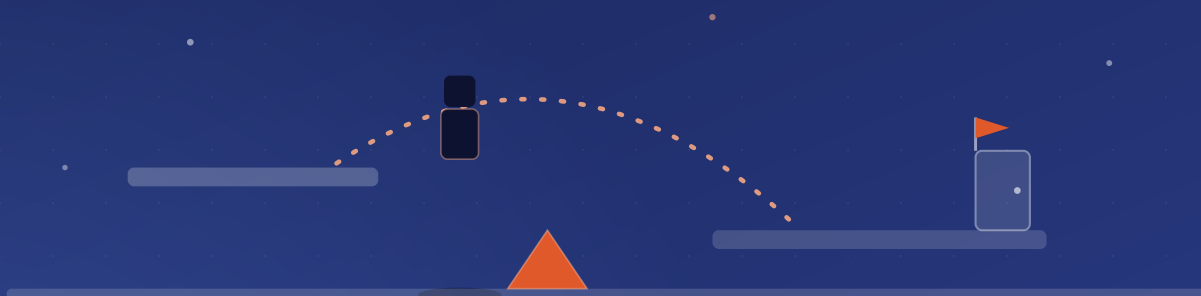


MOBILE 2D TROLL PLATFORMER

Unity 6 Mobile Troll Platformer Game Roadmap, Technical Architecture & Production Plan

A complete, implementation-focused blueprint for building an original minimalist pixel-art troll platformer — from pre-production through global launch and live operations.

Working title **PLUMMET** · *"It looks easy."*



ENGINE

Unity 6 (URP 2D Renderer)

PLATFORMS

Android & iOS (landscape)

ARCHITECTURE

Feature-first (Core + feature slices) · VContainer DI

AUDIENCE

Developers · Architects · Designers · Artists · QA · Producers

Table of Contents

PART I · GAME DESIGN & VISION	8
1 Executive Summary	8
1.1 Game overview	8
1.2 At a glance	8
1.3 Core gameplay loop	9
1.4 Target audience	9
1.5 Unique selling points	9
1.6 Commercial and production goals	9
1.7 Development team assumptions	9
1.8 Design summary	9
2 Game Vision and Design Pillars	10
2.1 Simple to understand	10
2.2 Hard to predict	10
2.3 Fast to retry	10
2.4 Funny to fail	10
2.5 Fair after discovery	10
2.6 Mobile-first	10
2.7 Minimalist visual style	11
2.8 Lightweight performance	11
2.9 Original trap mechanics	11
2.10 Short-session friendly	11
3 Core Gameplay Loop	11
3.1 Level start	12
3.2 Player observation	12
3.3 Movement and jumping	12
3.4 Trap discovery	12
3.5 Death or recovery	12
3.6 Instant restart	12
3.7 Learning through failure	12
3.8 Level completion	12
3.9 Progression to the next level	13
3.10 Why these choices	13
PART II · TECHNICAL DIRECTION & PRODUCTION MAP	13
4 Target Platform and Technical Direction	13
4.1 Platform targets	13
4.2 Runtime budgets (the contract)	13
4.3 Rendering: URP 2D Renderer	14
4.4 Input: the Input System package	14
4.5 Physics2D direction	14
4.6 Sprite and pixel-art import settings	14
4.7 Scene loading and Addressables	15
4.8 Services we will need	15
5 Feature Roadmap	15
5.1 Stage overview	15
5.2 Prototype	16

5.3 MVP	16
5.4 Vertical Slice	16
5.5 Alpha	16
5.6 Beta	17
5.7 Release Candidate	17
5.8 Global Release	17
5.9 Post-launch	17
PART III · PROJECT FOUNDATION & STRUCTURE	18
6 Unity 6 Project Setup	18
6.1 Creating the project	18
6.2 Recommended packages	18
6.3 Installing VContainer	19
6.4 2D rendering and Pixel Perfect Camera	19
6.5 Resolution, orientation, and safe areas	19
6.6 Physics2D configuration	20
6.7 Sprite and texture import settings	20
6.8 Sorting layers and collision layers	20
6.9 Quality settings	20
6.10 Android and iOS player settings	21
6.11 Addressables and Test Framework	21
6.12 Day-one setup checklist	21
7 Folder and File Hierarchy	21
7.1 Full project tree	22
7.2 What belongs in each major folder	23
7.3 Sample file hierarchies with real file names	24
8 Assembly Definition Strategy	25
8.1 The assemblies	26
8.2 Per-assembly contents and references	26
8.3 Reference rules (the contract)	27
8.4 Dependency diagram	27
8.5 Why this pays off	28
PART IV · CODE ARCHITECTURE	28
9 Industry-Level Code Architecture	29
9.1 The shape: Core, Features, App	29
9.2 The two rules that hold it together	29
9.3 Who may reference whom	30
9.4 Architecture diagram	30
9.5 Keeping the design honest	30
10 Dependency Injection with VContainer	31
10.1 Why VContainer	31
10.2 Scopes and feature installers	31
10.3 A feature installer	32
10.4 The root composition root	33
10.5 The level scope and entry points	34
10.6 IStartable, ITickable, IFixedTickable — and not overusing them	36
10.7 Injecting into MonoBehaviours and avoiding service-locator misuse	36
11 Recommended Design Patterns	36

11.1	Pattern catalogue	36
11.2	State pattern interfaces	37
11.3	The game states	38
12	Scene Architecture	38
12.1	Scene responsibilities	39
12.2	What persists, what is recreated	39
12.3	How restart works	39
12.4	Avoiding DontDestroyOnLoad abuse	39
12.5	Scene and state flow	39
13	Game State Flow	40
PART V · CORE GAMEPLAY SYSTEMS		41
14	Player System	41
14.1	Responsibilities and split	41
14.2	Movement features	41
14.3	Input abstraction	41
14.4	Tuning via ScriptableObject	42
14.5	Movement service outline	42
14.6	Animation states and tuning	43
15	Trap System Architecture	43
15.1	The original trap catalogue	43
15.2	How traps work	44
15.3	Trap interface (Core.Contracts)	44
15.4	Trap base class (Features.Traps)	44
15.5	Example: falling platform trap	45
15.6	Avoiding hardcoded one-off level scripts	46
16	Level Design Pipeline	46
16.1	Construction rules	46
16.2	Fast-restart design	47
16.3	Fifty-level progression plan	47
PART VI · AUDIO, INTERFACE & ART DIRECTION		48
17	Audio and Haptics	48
17.1	Sound set	48
17.2	Haptics	49
17.3	Audio architecture and mobile constraints	49
18	UI/UX Design	49
18.1	Screens and elements	49
18.2	Touch targets and placement	50
18.3	Fast-restart UX and friction reduction	50
18.4	Accessibility	50
19	Art Direction	50
19.1	Style summary	50
19.2	Technical art specs	51
19.3	Color palette and readability	51
19.4	Character direction (reference)	51
PART VII · DATA, PERFORMANCE & QUALITY		51

20	Save System	52
20.1	What is saved	52
20.2	Save data model (Core.Contracts)	52
20.3	Save service interface (Core.Contracts)	52
20.4	Versioning, migration, and corruption handling	53
21	Mobile Optimization Techniques	53
21.1	Rendering and textures	53
21.2	Memory and allocation	54
21.3	Physics and simulation	54
21.4	UI, audio, and content streaming	54
21.5	Profiling and verification	54
21.6	Performance budget	55
21.7	Object pool interface (Core.Contracts)	55
21.8	Pooled effect service outline	55
22	Analytics and Telemetry	56
22.1	Event set	56
22.2	How the data drives design	56
22.3	Analytics service interface (Core.Contracts)	56
23	Testing Strategy	57
23.1	Test types	57
23.2	Example EditMode test	57
24	Debug and Developer Tools	58
24.1	Tooling	58
24.2	Editor-only vs. development build	59
PART VIII · MONETIZATION, RELEASE & COMPLIANCE		59
25	Ads and Monetization	59
25.1	Revenue options	59
25.2	Ethical monetization rules (hard constraints)	59
26	Build and Deployment Pipeline	60
26.1	Platform builds	60
26.2	Signing and identifiers	60
26.3	Versioning and build numbers	60
26.4	Testing tracks and release flow	60
26.5	CI/CD	61
26.6	Crash reporting and store assets	61
26.7	Release checklist	61
27	Privacy, Compliance, and Store Requirements	61
27.1	Requirements	62
PART IX · PRODUCTION MANAGEMENT		62
28	Production Timeline	63
28.1	Phases	63
28.2	Milestone table	63
29	Team Roles and Responsibilities	65
29.1	Roles	65
29.2	Solo developer configuration	65
30	Risk Management	66

PART X · ENGINEERING STANDARDS & PROCESS	67
31 Coding Standards	67
31.1 Naming conventions	67
31.2 Namespace and file organisation	68
31.3 MonoBehaviour rules	68
31.4 ScriptableObject rules	68
31.5 Events, delegates, and messaging	69
31.6 Asynchronous code	69
31.7 Error handling	69
31.8 Logging	69
31.9 Comments and XML documentation	70
31.10 Testing expectations	70
31.11 Architectural rules — the non-negotiables	70
31.12 Code review checklist	71
32 Git Workflow	71
32.1 Repository structure	72
32.2 Git LFS for binary assets	72
32.3 Branching model	72
32.4 Commit messages	73
32.5 Pull requests	73
32.6 PR review checklist	74
32.7 Tags, releases, and ignore rules	74
33 Documentation Strategy	75
33.1 Where documentation lives	75
33.2 README	75
33.3 Core guides	76
33.4 Checklists	76
33.5 Known issues and troubleshooting	76
33.6 Architecture Decision Records (ADRs)	76
33.7 Keeping documentation alive	77
 PART XI · REFERENCE & APPENDICES	 77
34 Final Production Checklist	77
34.1 Code	77
34.2 Gameplay	78
34.3 Art	78
34.4 Audio	78
34.5 Mobile performance	79
34.6 Monetization	79
34.7 Release	79
35 Appendices	79
A Full Folder and File Tree	80
B Assembly Dependency Map	80
C VContainer Lifetime-Scope Reference	81
D Service Interface Reference	81
E Trap Configuration Example	82
F Level Data Example	83
G Mobile Optimization Checklist	84

H	QA Checklist	85
I	Release Checklist	85
J	Glossary	86

This document is an original production and technical-architecture plan. All characters, traps, level designs, mechanics, art direction, and branding are original. Page references are clickable; bookmarks mirror the heading hierarchy.

Part I · Game Design & Vision

1. Executive Summary

PLUMMET (working title) is a mobile-first 2D troll platformer for Android and iOS, built in **Unity 6**. Each level is presented as an almost insultingly simple room: a blocky silhouette character, a few flat platforms, a door at the far end. The player's only job is to walk to the door. The joke is that the room is lying. Floors fall away, spikes erupt on the second step, the door slides backwards as you approach, the "victory" screen drops you into a pit. Players die, laugh, restart in a quarter of a second, and try again — now armed with the one piece of knowledge the level was hiding.

The promise in one line: *It looks easy.* The entire product is built to set that expectation and then break it in a way that is surprising the first time and fair every time after.

1.1 Game overview

The game ships as a linear campaign of short, single-screen (or short side-scrolling) levels. A level lasts five to twenty seconds once solved and frequently kills the player two or three times before they read the trick. Failure is never punished with lost progress — the player respawns instantly at the level start. Progression is gated only by completing the previous level, and a level-select grid lets players revisit anything they have unlocked.

The product is deliberately small in surface area and deep in *content volume*: one tight movement model, one reusable trap framework, and a very large number of hand-authored levels that recombine a fixed catalogue of traps in fresh, deceptive ways.

1.2 At a glance

Attribute	Decision
Genre	Single-screen / short-scroll 2D troll platformer
Engine	Unity 6 (Universal Render Pipeline, 2D Renderer)
Platforms	Android and iOS, landscape orientation
Art style	Minimalist pixel art, flat silhouettes, muted gradient backgrounds
Session length	30 seconds to a few minutes; ideal for short bursts
Architecture	Feature-first architecture (Core + feature slices) with VContainer dependency injection
Monetization	Optional rewarded ads, light interstitials, remove-ads IAP, cosmetic skins
Primary KPI	D1/D7 retention, levels-per-session, completion funnel per level

1.3 Core gameplay loop

Observe a simple-looking room → move and jump toward the door → trigger a hidden trap → die (funny) or adapt → restart instantly → apply what you learned → reach the door → unlock the next level. Sections 3 and 13 expand this loop into systems and states.

1.4 Target audience

The audience is broad and casual-leaning, skewing toward players who enjoy "rage" and trick games, short-form-video culture, and shareable failure clips. The control scheme assumes a phone held in two hands in landscape. No prior platformer skill is required; the difficulty is *knowledge*, not execution.

1.5 Unique selling points

- **Deception as the core mechanic.** The challenge is figuring out the lie, not pixel-perfect jumps.
- **Failure is the content.** Deaths are fast, funny, and screenshot/clip-friendly — naturally shareable.
- **Instant restart.** Sub-0.3-second respawn keeps players in flow and removes the sting of dying.
- **Minimalist clarity.** The plain art style is a design tool: it makes every trap *look* fair in hindsight.
- **Original trap catalogue.** A reusable, data-driven trap system lets designers compose surprises without one-off scripting.

1.6 Commercial and production goals

The **commercial goal** is a high-retention, low-friction free-to-play title monetized ethically (Section 25), with content updates (new level packs, daily challenges, skins) sustaining a long tail. The **production goal** is to ship a polished vertical slice quickly, validate retention, then scale level count aggressively because the trap framework makes new levels cheap to author.

1.7 Development team assumptions

This document assumes a small indie team of one to four people (Section 28 describes both the multi-person and solo configurations). It assumes Unity 6, source control with Git + Git LFS, a CI runner for builds, and a willingness to adopt dependency injection and assembly-definition discipline from day one rather than retrofitting it.

Originality note. All characters, traps, level layouts, mechanics, art, and branding in this document are original. *PLUMMET*, the silhouette hero *Blip*, and all trap names are working names. Run a trademark and store-name availability search before any public listing — a cheap step that avoids costly post-launch rebrands.

1.8 Design summary

The game must be, in priority order: **easy to understand**, **difficult to predict**, **fast to retry**, **funny to fail**, **satisfying to solve**, and **optimized for mobile**. Every system decision in this document is justified against that list.

2. Game Vision and Design Pillars

The creative vision is a game that looks like a five-year-old drew it and plays like it is actively trying to trick you — and yet, after you die, you always agree the trick was fair. The visuals are plain and readable on purpose: minimalism is what makes the surprises legible in hindsight. If the screen were busy, a hidden spike would feel like noise; because the screen is empty, the spike feels like a *reveal*.

The ten pillars below are the contract for every feature, level, and asset. When a decision is ambiguous, resolve it in favor of the higher-priority pillar.

2.1 Simple to understand

A new player must grasp the goal in under three seconds with zero text: there is a character, there is a door, walk to the door. Controls are two movement buttons and a jump. No tutorials, no stat screens, no inventory. The first three levels teach movement and the *existence* of traps without words.

2.2 Hard to predict

Every level hides at least one deception the player cannot infer from a glance. Predictability is the enemy: once a trap pattern becomes obvious, it must be remixed (delayed, mirrored, chained, or inverted) so the player's pattern-matching is repeatedly subverted. The trap framework (Section 15) exists precisely to make remixing cheap.

2.3 Fast to retry

Death-to-playable must stay under 0.3 seconds (Section 19, Section 21). No death animation longer than a beat, no "you died" interstitial, no loading. The player is back at the start before frustration can form. Fast retry is what converts "rage" into "one more try."

2.4 Funny to fail

Failure is the product's main shareable moment. Deaths are exaggerated and a touch slapstick: an over-the-top squish, a comedic sound sting, a fake victory that yanks the rug. The tone is mischievous, never mean. We never mock the player; the *level* is the prankster.

2.5 Fair after discovery

The first death may be unavoidable, but the trick must always be learnable from that single death. After the player knows the secret, the level must be beatable with ordinary execution. No trap may require luck, frame-perfect timing the player can't see, or knowledge that isn't visible on a second attempt.

2.6 Mobile-first

Designed for touch in landscape, for short sessions, for one-handed-friendly menus and two-handed play. Button placement, hit-target sizing, and battery/thermal behavior are first-class constraints (Sections 4, 18, 19, 21), not afterthoughts ported from a hypothetical PC build.

2.7 Minimalist visual style

Flat silhouettes, simple rectangles, muted gradients, strong contrast, minimal visual noise (Section 17). The style is cheap to produce, fast to render, and — crucially — a gameplay tool that keeps traps readable.

2.8 Lightweight performance

The game targets low-end Android. It must hold a stable frame rate, load levels in under a second, restart instantly, and keep a small build and memory footprint (Section 21). Simplicity in art and architecture serves performance directly.

2.9 Original trap mechanics

Traps are designed as an original, named catalogue (fake floors, popup spikes, fleeing doors, control inversions, fake victories, and more — Section 15) implemented through one reusable system. We never copy another game's specific levels; we build a generative toolkit and author original combinations.

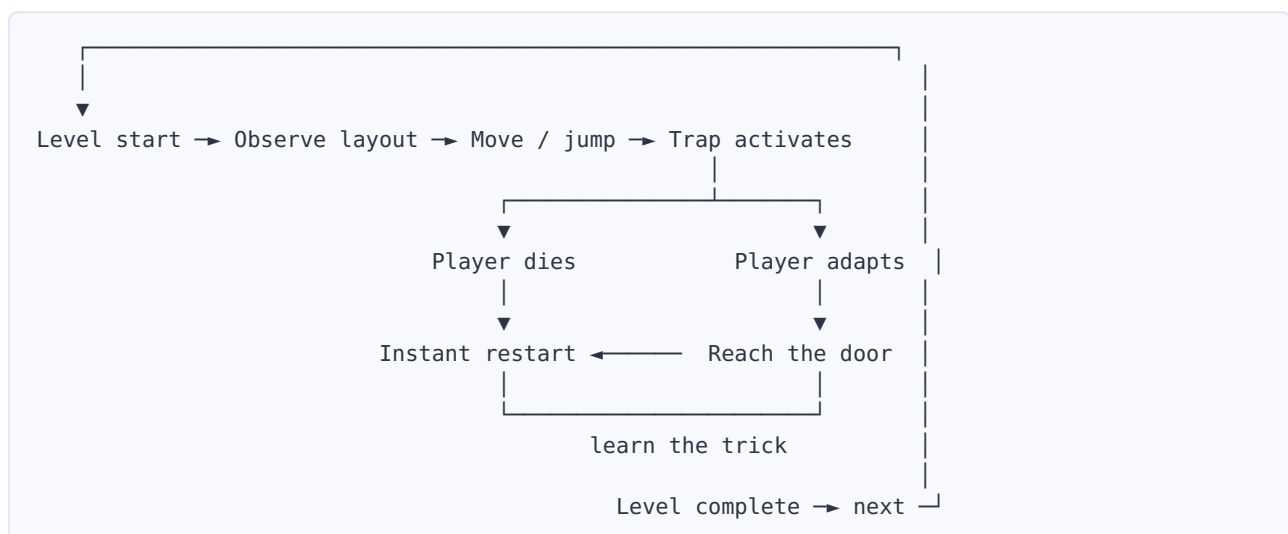
2.10 Short-session friendly

The game respects the player's time. Levels are bite-sized, progress saves continuously, and the game suspends and resumes cleanly when a call or notification interrupts play (Section 13's `AppSuspended` state). It is designed to be picked up for ninety seconds and put down without losing anything.

3. Core Gameplay Loop

The loop is the heartbeat of the game. It must feel tight enough that "just one more" is the default reaction to dying. This section describes the loop as a player experiences it; Section 13 formalizes it as a state machine and Section 14 implements the movement underneath it.

The moment-to-moment loop



3.1 Level start

The level fades in fast (under ~150 ms) with the character idle at the spawn point and the door visible. No countdown. Input is enabled immediately. The camera is static or gently framed so the player can take in the whole room.

3.2 Player observation

For a beat, the player reads the room. The minimalist art means there is little to read — which is the trap. The player forms a (usually wrong) mental model: "I just walk right and jump that gap." This false confidence is the setup for the prank.

3.3 Movement and jumping

The player uses left/right and jump. Movement must feel responsive and forgiving (coyote time, jump buffering — Section 14) so that when the player dies, they blame the *trap*, not the controls. Floaty or unfair controls would violate Pillar 2.5 and poison the whole experience.

3.4 Trap discovery

Somewhere between the spawn and the door, the level springs its surprise: the floor vanishes, a spike pops, the door flees, the controls reverse. The first encounter is designed to land *after* the player has committed to a motion, so the reveal feels like a gotcha rather than a warning.

3.5 Death or recovery

Most first encounters end in death. A well-designed trap occasionally allows a quick-reflex recovery, which feels great and rewards attentive players. Death triggers a short, funny animation and sound, then immediately routes to restart.

3.6 Instant restart

The level resets to its initial state — character at spawn, every trap reset (Section 15's `ResetTrap`), camera reframed — with no perceptible load. This is a hard technical requirement (Section 21): we reset objects in place rather than reloading scenes.

3.7 Learning through failure

Each death teaches exactly one fact. The player internalizes "the third tile is fake" or "jump *before* the door." The loop is a compressed teach-test cycle: the level poses a question, the player answers wrong, learns the answer, and proves it.

3.8 Level completion

Reaching the (real) door plays a satisfying completion beat — a short sound, a small celebratory effect — and transitions to the next level. Completion should feel earned precisely because the level tried so hard to stop the player.

3.9 Progression to the next level

The next level unlocks and loads. Save state updates (Section 20). Difficulty and novelty ramp according to the level-design curve (Section 16). Occasionally the *next* level weaponizes the lesson the player just learned, punishing the habit the previous level taught — a deliberate, delightful betrayal.

3.10 Why these choices

Why fast restart is non-negotiable. Friction after death (animations, ads, loads) is where players quit. Removing it converts deaths into momentum. **Why levels are short.** Short levels mean a death costs seconds, not minutes, so trial-and-error is cheap and the teach-test cycle stays tight. **Why failure must be funny, not unfair.** Funny failure is shareable and keeps the mood light; unfair failure (Pillar 2.5 violations) reads as a bug and breaks trust. **How surprise and fairness are balanced.** Surprise comes from hiding information on the first attempt; fairness comes from guaranteeing that information is visible and the challenge is executable on every attempt after. The designer's job (Section 16) is to keep those two forces in tension without letting either win.

Part II · Technical Direction & Production Map

4. Target Platform and Technical Direction

This section fixes the technical envelope: what we build for, with what rendering and input stack, and what the non-negotiable runtime budgets are. Decisions here constrain every later section.

4.1 Platform targets

We ship **Android** (primary, because of reach and the low-end long tail) and **iOS**. We design and test for low-end Android first; if it runs well there, it runs everywhere. The orientation is **landscape**, locked, because two-handed landscape gives comfortable space for left/right buttons on one side and jump on the other.

Platform	Min target	Notes
Android	Android 8.0 (API 26), 2 GB RAM class devices	Primary perf target; test on a real budget device
iOS	iOS 15+, iPhone 8 and newer	Secondary; generally over-powered for this game
Orientation	Landscape (locked)	Two-handed control layout
Aspect ratios	16:9 up to 20:9, plus tablet 4:3	Safe-area handling for notches/home indicator

4.2 Runtime budgets (the contract)

These are commitments, not aspirations. Section 21 explains how to hold them; Section 22 tests them.

Budget	Target	Hard ceiling
Frame rate	60 FPS	30 FPS fallback on low-end
Restart (death → playable)	< 0.20 s	0.30 s

Budget	Target	Hard ceiling
Level load (select → playable)	< 0.6 s	1.0 s
Memory (low-end Android)	< 200 MB	250 MB
Build size	< 80 MB	100 MB
Draw calls (gameplay)	< 40	80

4.3 Rendering: URP 2D Renderer

Use the **Universal Render Pipeline with the 2D Renderer**, not the Built-in pipeline. URP 2D is the actively-developed 2D path in Unity 6, gives us the 2D renderer features (sprite sorting, optional 2D lights if we ever want them), SRP Batcher benefits, and clean per-platform quality control. For a flat-sprite game we keep post-processing off, lights off (or a single global light), and rely on sprite atlasing (Section 21) to keep draw calls minimal.

Decision: URP 2D Renderer. Rationale: future-proof, SRP Batcher, per-quality-level control, and no need for Built-in's legacy baggage. Keep the URP asset minimal — no HDR, no MSAA on low-end, no post stack.

4.4 Input: the Input System package

Use the new **Input System** package (not legacy `Input`). It cleanly abstracts touch, supports our on-screen control scheme, and lets us define an input *action map* that our `IPlayerInput` abstraction (Section 14) reads from. On-screen buttons feed the action map via the On-Screen Control components, so the same actions work with a controller or keyboard in the editor for fast iteration.

4.5 Physics2D direction

Movement uses `Rigidbody2D` in a controlled way. The player is a `Dynamic` body with gravity, but we drive horizontal motion deterministically and clamp velocities so behavior is predictable (Pillar 2.5). Static level geometry uses `Static` colliders; falling platforms flip from `Kinematic` to `Dynamic` on trigger (Section 15). We configure a tight **collision matrix** (Section 21) so the player only tests against the layers it must.

Key Physics2D settings: a fixed timestep of `0.0166667` (60 Hz) to match the frame target, gravity tuned per the movement config rather than left at default, and `Queries Hit Triggers` enabled so trap trigger zones register.

4.6 Sprite and pixel-art import settings

All sprites import with: **Filter Mode = Point (no filter)**, **Compression = None** (for crisp pixels) or a high-quality setting, **Pixels Per Unit (PPU)** standardized project-wide (Section 17 recommends 16 PPU for the body unit), **Mesh Type = Full Rect**, and **no mip maps** for UI/gameplay sprites that never scale down in 3D space. A **Pixel Perfect Camera** component (Section 6) locks the pixel grid so art never shimmers or sub-pixel-crawls.

4.7 Scene loading and Addressables

We keep a small fixed set of scenes (`Boot`, `MainMenu`, `Loading`, `Game` — Section 12). Levels themselves are **prefabs / data**, not scenes, so switching levels is an in-place reset, not a scene reload (this is how we hit the restart budget). **Addressables** package level packs, sprite atlases, audio, and UI so we can ship a small core build and download/patch content packs later (Section 21).

4.8 Services we will need

Concern	Stack choice
Save	Local JSON to <code>Application.persistentDataPath</code> , versioned (Section 20); optional cloud later
Analytics	A vendor-agnostic <code>IAalyticsService</code> (Section 22) wrapping the chosen SDK
Ads	A vendor-agnostic ads service wrapping a mediation SDK (Section 25)
Crash reporting	A crash/error SDK initialized in <code>Boot</code> (Section 26)
DI	VContainer (Section 10)

The pattern is identical for analytics, ads, and crash reporting: define a contract in `Game.Core.Contracts`, implement it as a service in `Game.Core` behind that contract, register it in a VContainer scope. This keeps third-party SDKs swappable and keeps every feature ignorant of vendor specifics.

5. Feature Roadmap

The game is built in eight production stages, from a throwaway prototype to live operations. Each stage has a content target and a clear exit gate; do not start the next stage until the current gate is met. Section 27 maps these stages onto a calendar.

5.1 Stage overview

Stage	Content target	Headline goal	Exit gate
Prototype	1 test level	Prove the feel and the joke	Movement + one trap + restart feel fun
MVP	10 levels	Prove the systems	Core loop, save, basic UI working end-to-end
Vertical Slice	15 polished levels	Prove the look and retention	One pack at shippable quality on device
Alpha	40+ levels	Feature complete	All systems present; analytics live; QA tools in
Beta	70+ levels	Content + polish complete	Ads integrated; perf in budget; store-ready
Release Candidate	Final	Ship-quality	All blockers closed; store assets done
Global Release	Live	Launch	Live on both stores; monitoring on

Stage	Content target	Headline goal	Exit gate
Post-launch	Ongoing	Grow + retain	Recurring updates shipping

5.2 Prototype

- Basic player movement and jumping
- One hand-built test level
- Spikes and a completion door
- Restart system (in-place reset)
- Simple on-screen mobile controls

Purpose: answer one question — is walking into a lie and dying *fun*? If the prototype isn't fun with one trap, more content won't save it.

5.3 MVP

- 10 playable levels
- Basic trap **trigger** system (the reusable framework's first cut)
- Fake floor, falling platform, hidden spike traps
- Level progression and unlock
- Basic UI (play, restart, level-complete)
- Basic save system (current/highest level)

Purpose: validate that the systems compose — that new levels are authored from data and reused traps, not bespoke scripts.

5.4 Vertical Slice

- 15 polished levels representing final quality
- Final visual direction sample (Section 17 applied for real)
- Final player movement feel (tuning locked)
- Audio sample pass (Section 21)
- Main menu and level selection
- Several trap types in rotation
- Tight restart loop
- Performance test on a real low-end device

Purpose: this is the artifact you show publishers and run a small retention test on. It must look and feel like the finished game.

5.5 Alpha

- 40+ levels
- Expanded trap catalogue
- Game state system complete (Section 13)
- Analytics events firing (Section 22)

- Settings and pause menus
- In-editor/dev QA and debug tools (Section 25 of QA; debug tools Section 25)
- Device testing matrix started

Purpose: feature-complete. After Alpha, no new *systems* — only content, polish, and fixes.

5.6 Beta

- 70+ levels (target 100 by launch)
- Polished UI across all screens
- Full audio pass
- Ads integration (Section 25)
- Save versioning and migration (Section 20)
- Performance optimization pass to budget
- Store metadata preparation

Purpose: content-complete and store-ready. Beta is where the long content tail gets built quickly thanks to the trap framework.

5.7 Release Candidate

- Final bug fixing (blocker/critical only)
- Final QA pass against the Section 34 checklist
- Store screenshots and app icon
- Privacy policy published (Section 33)
- Build signing configured (Section 26)
- TestFlight build live
- Google Play internal testing track live

Purpose: a build you would ship, held for final validation only.

5.8 Global Release

- Android production release
- iOS production release
- Analytics dashboards monitored
- Crash monitoring active
- Hotfix plan and on-call owner defined (Section 26)

5.9 Post-launch

- New level packs (cheap to author — the payoff of the framework)
- Daily challenge mode
- Cosmetic skins for *Blip*
- Leaderboards (fastest clears, fewest deaths)
- Seasonal trap variants
- A/B testing of ad frequency and onboarding (Section 25)

Purpose: retention and revenue tail. The architecture (Parts III–IV) is what makes shipping this steady stream of content low-risk.

Part III · Project Foundation & Structure

This part is deliberately placed **before** any architecture or implementation discussion. A team that agrees on the project layout, assembly boundaries, and Unity settings on day one avoids the single most expensive class of refactor: moving code across assemblies after dependencies have calcified. Set this up first, commit it, and build everything else inside it.

6. Unity 6 Project Setup

This section is a concrete, do-this-in-order setup checklist for a fresh Unity 6 project targeting Android and iOS. The goal is a small, fast, deterministic 2D project with the Input System, URP 2D Renderer, and dependency-injection tooling in place from the first commit.

6.1 Creating the project

Create the project from the **Universal 2D** template in Unity Hub on the latest Unity 6 LTS release. The 2D template wires up the URP 2D Renderer, a 2D-appropriate **URP-Performant** asset, and 2D physics defaults, which is closer to the target than the empty 3D template. Set the project name to the studio convention (for example **Plummet**), pick a versioned Unity install, and enable version control as **Visible Meta Files** with **Force Text** serialization — both are required for clean Git diffs and merges.

Editor settings to set immediately. In *Edit* ► *Project Settings* ► *Editor: Asset Serialization* → **Force Text**; Version Control → **Visible Meta Files**; Enter Play Mode Options → **enabled** with *Reload Domain* and *Reload Scene* **off** for faster iteration (your code must therefore not rely on static state surviving or resetting between play sessions — which the architecture in Part IV already enforces by avoiding static managers).

6.2 Recommended packages

Install via *Window* ► *Package Manager*. Keep the set minimal; every package is build size, surface area, and a potential upgrade headache.

Package	Purpose	Notes
Input System	All player and UI input	Replaces legacy Input; set Active Input Handling to <i>Input System Package</i>
Universal RP	2D Renderer, rendering	Comes with the 2D template
2D Sprite / 2D Common	Sprite editing, atlasing	Sprite Atlas lives here
TextMeshPro	All text rendering	Import Essentials on first use

Package	Purpose	Notes
Addressables	Level/asset streaming, build layout	Used for level packs (Section 12, Section 21)
Unity Test Framework	EditMode + PlayMode tests	Required for Section 23
VContainer	Dependency injection	Install via UPM Git URL or OpenUPM (Section 10)
Burst + Mathematics	Optional, only if hot math paths appear	Do not add speculatively
Mobile Notifications	Optional, only if used	Defer until a feature needs it

Analytics, Ads, and Crash Reporting SDKs are added later, behind interfaces, during Alpha/Beta (Sections 22, 25, 26). They are intentionally **not** installed at setup so that early prototype builds stay clean and dependency-free.

6.3 Installing VContainer

VContainer is distributed as a UPM package. Add it through *Package Manager* ► ► *Add package from git URL* (or via OpenUPM). After install, no global bootstrapping is needed beyond creating `LifetimeScope` components — VContainer hooks into the player loop automatically through `RegisterEntryPoint`. Pin the version in `Packages/manifest.json` and commit `Packages/packages-lock.json` so every machine and the CI runner resolve the exact same version.

Adding VContainer via the Package Manager git URL

```
https://github.com/hadashiA/VContainer.git?path=VContainer/Assets/VContainer#vX.Y.Z
```

Replace `vX.Y.Z` with the version pinned for the project. Treat dependency-injection framework upgrades like engine upgrades: scheduled, tested, never silent.

6.4 2D rendering and Pixel Perfect Camera

The visual target is crisp, stable pixel art (Section 19). Configure the URP 2D Renderer asset and add a **Pixel Perfect Camera** component to the main camera.

- Set a project-wide **Pixels Per Unit (PPU)** of **16** and author all sprites to that PPU (Section 19).
- On the Pixel Perfect Camera: set **Assets Pixels Per Unit** to 16, set a fixed **Reference Resolution** (for example `640 × 360`, a 16:9 box that scales cleanly to common phone aspect ratios), enable **Crop Frame** as needed, and enable **Pixel Snapping**.
- Disable anti-aliasing in the URP asset (MSAA off) — it blurs pixel art and costs fill rate on mobile.
- Set the camera to **Orthographic**.

6.5 Resolution, orientation, and safe areas

Lock orientation to **Landscape** (both left and right) in *Project Settings* ► *Player* ► *Resolution and Presentation*. Design the HUD against the **safe area** (notches, punch-holes, gesture bars) using a safe-area anchor utility on the root canvas (Section 18). Target a reference aspect of 16:9 but verify layout

from 4:3 (older tablets) through 20:9 (tall phones); the gameplay camera shows slightly more or less horizontal space, which level design must tolerate (Section 16).

6.6 Physics2D configuration

Troll-platformer feel depends on deterministic, tight physics. Configure *Project Settings* ► *Physics 2D*:

- **Fixed Timestep** (in *Time* settings) at `0.0166667` (60 Hz) so physics matches the 60 FPS target; never run movement off `Update` delta alone (Section 14).
- Set **Gravity** Y to a tuned value (start around `-30` to `-40` in project units for snappy arcs; final value lives in `PlayerMovementConfig`, Section 14).
- Define the **Layer Collision Matrix** so only meaningful pairs collide (Player ↔ Solid, Player ↔ Trap, Player ↔ Trigger). Disable all other pairs to cut broadphase work (Section 21).
- Set **Queries Hit Triggers** appropriately and prefer **Simulation Mode: Fixed Update**.

6.7 Sprite and texture import settings

Author and import all gameplay art as pixel art:

- **Texture Type:** Sprite (2D and UI). **Sprite Mode:** Single or Multiple (atlas pages).
- **Filter Mode: Point (no filter)**. **Compression: None** for small pixel sprites (compression artifacts are visible at this scale); rely on atlasing to keep memory and draw calls low.
- **Pixels Per Unit:** 16, matching the project. **Mesh Type:** Full Rect. **Generate Mip Maps: off** for 2D sprites.
- Group sprites into **Sprite Atlases** per category (characters, platforms, traps, UI) so the renderer can batch (Section 21).

6.8 Sorting layers and collision layers

Define **Sorting Layers** (render order) and physics **Layers** (collision) explicitly and document them. Suggested baseline:

Concern	Layer type	Entries
Render order	Sorting Layers	<code>Background</code> , <code>Platforms</code> , <code>Traps</code> , <code>Player</code> , <code>VFX</code> , <code>UI</code>
Collision	Physics Layers	<code>Solid</code> , <code>OneWay</code> , <code>Trap</code> , <code>Trigger</code> , <code>Player</code> , <code>Killbox</code>

Keeping these small and named (never magic integers in code) is part of the coding standards (Section 31) and directly supports collision-matrix optimization (Section 21).

6.9 Quality settings

Create a **single mobile quality level** (delete unused Low/Medium/High tiers to avoid accidental divergence). Disable shadows (2D, no need), set **VSync** off and control the frame rate explicitly via `Application.targetFrameRate` (Section 21), disable HDR in the URP asset, and keep the URP asset's renderer feature list empty unless a feature is justified.

6.10 Android and iOS player settings

Configure both platforms now so build breakage is caught early, even if you only build Android during prototype.

- **Android:** Scripting Backend **IL2CPP**, Target Architectures **ARM64** (and ARMv7 only if a market requires it), **Minimum API Level** set to a low-end-inclusive but currently store-compliant level, **Target API Level** to the latest store-required level, package name `com.[studio].plummet`, and **Managed Stripping Level** Medium to start. Texture compression default **ASTC**.
- **iOS:** **Target minimum iOS version** set to a currently supported floor, bundle identifier `com.[studio].plummet`, **Camera/Tracking usage descriptions** only if a feature requires them, and architecture ARM64. Plan ahead for App Tracking Transparency if ads use the advertising identifier (Section 27).
- Set **Color Space** to **Linear** if the art direction relies on it, but **Gamma** is acceptable and cheaper for flat pixel art — decide once and document it.

6.11 Addressables and Test Framework

Initialize Addressables (*Window ▶ Asset Management ▶ Addressables ▶ Groups ▶ Create Settings*) and create groups that mirror content categories — `Levels`, `Sprites`, `Audio`, `UI` — so level packs can ship as separately downloadable or separately updatable content later (Section 21, Section 25). Enable the Test Framework by adding test assemblies (Section 8) and create the EditMode and PlayMode assembly definitions referenced in Section 23.

6.12 Day-one setup checklist

- ☐ Project created from Universal 2D template on pinned Unity 6 LTS; Force Text + Visible Meta Files enabled.
- ☐ Required packages installed; `packages-lock.json` committed.
- ☐ VContainer installed and version-pinned.
- ☐ URP 2D Renderer + Pixel Perfect Camera configured at 16 PPU, MSAA off, orthographic.
- ☐ Orientation locked to landscape; safe-area handling planned.
- ☐ Physics2D fixed timestep at 60 Hz; collision matrix trimmed.
- ☐ Sprite import defaults set to Point filter, no mips, atlased.
- ☐ Sorting layers and physics layers defined and named.
- ☐ Single mobile quality level; explicit frame-rate control.
- ☐ Android (IL2CPP/ARM64) and iOS player settings configured and both platforms build.
- ☐ Addressables groups created; test assemblies in place.
- ☐ First commit contains the empty-but-correct project skeleton.

7. Folder and File Hierarchy

A consistent folder structure is the cheapest form of architecture enforcement: if a file has an obvious home, it lands in the right assembly and dependencies flow correctly almost by accident. This project is organised **feature-first** (Section 9): code is grouped by *what the player experiences* — Player, Traps,

Levels, Progression, UI — rather than by technical layer. A small **Core** holds the shared contracts and cross-cutting services every feature builds on, and a single **App** assembly composes everything. Each top-level code folder maps cleanly to an assembly definition (Section 8).

7.1 Full project tree

Assets/_Project — complete folder hierarchy (feature-first)

```
Assets/
  _Project/
    Art/
      Characters/      # Blip silhouette sprites, animation frames
      Environments/    # Gradient backgrounds, decorative layers
      Platforms/       # Solid, one-way, moving platform sprites
      Traps/           # Spike, falling-block, fake-door sprites
      UI/              # Buttons, icons, font textures
      VFX/             # Death puff, dust, impact sprites

    Audio/
      Music/           # Menu loop, ambient beds
      SFX/             # Jump, land, death, trap, UI stingers

    Code/
      Core/             # Shared foundation – no feature deps
      Contracts/        # Game.Core.Contracts (engine-light API surface)
      Save/             ISaveService.cs, SaveData.cs, SaveVersion.cs
      Analytics/        IAnalyticsService.cs
      Ads/              IAdsService.cs
      Audio/            IAudioService.cs
      Input/            IPlayerInput.cs
      Scenes/           ISceneLoader.cs
      Pooling/          IObjectPool.cs
      Events/           ISignalBus.cs, Signals/ (PlayerDiedSignal, LevelCompletedSignal...)
      State/            IGameState.cs, IGameStateService.cs, GameState.cs
      Traps/            ITrap.cs, TrapConfig.cs # shared so Levels can place traps
      Runtime/          # Game.Core (concrete shared services + DI scaffolding)
      DI/               RootLifetimeScope.cs, IFeatureInstaller.cs
      Save/             JsonSaveService.cs, SaveMigrationService.cs
      Analytics/        AnalyticsService.cs
      Ads/              AdsService.cs
      Audio/            AudioService.cs
      Input/            MobilePlayerInput.cs
      Scenes/           UnitySceneLoader.cs
      Pooling/          ObjectPool.cs, EffectPoolService.cs
      Events/           SignalBus.cs
      State/            GameStateService.cs, States/ (BootState, PlayingState...)
      Common/           guards, result types, extensions, base classes

      Features/         # One self-contained vertical slice per folder
      Player/           # Game.Features.Player
      Runtime/          PlayerView.cs, PlayerMovementService.cs, PlayerGroundChecker.cs,
                        PlayerDeathHandler.cs, PlayerAnimator.cs
      Config/           PlayerMovementConfig.cs
      DI/               PlayerInstaller.cs
      Tests/            PlayerMovementTests.cs # Game.Features.Player.Tests
      Traps/            # Game.Features.Traps
      Runtime/          TrapBase.cs, TrapTrigger.cs, FallingPlatformTrap.cs,
                        SpikeTrap.cs, FakeFloorTrap.cs, FakeDoorTrap.cs,
                        ReverseControlsTrap.cs
      Config/           FallingPlatformTrapConfig.cs, SpikeTrapConfig.cs ...
      DI/               TrapsInstaller.cs
      Tests/            TrapStateTests.cs # Game.Features.Traps.Tests
```

```

Levels/                                     # Game.Features.Levels
  Runtime/   LevelLoader.cs, LevelRuntime.cs, TrapResetService.cs
  Data/      LevelData.cs, TrapPlacement.cs
  DI/        LevelsInstaller.cs
  Tests/     LevelProgressTests.cs          # Game.Features.Levels.Tests
Progression/ # Game.Features.Progression (logic-only, engine-free)
  Runtime/   ProgressionService.cs, UnlockRules.cs
  DI/        ProgressionInstaller.cs
  Tests/     ProgressionTests.cs           # Game.Features.Progression.Tests
Ui/          # Game.Features.Ui
  Runtime/   HudPresenter.cs, HudView.cs, MainMenuPresenter.cs, MenuView.cs,
             LevelSelectPresenter.cs, SettingsPresenter.cs, PausePresenter.cs
  DI/        UiInstaller.cs
  Tests/     HudPresenterTests.cs          # Game.Features.Ui.Tests

App/                                     # Game.App – the single composition root
Bootstrap/  GameBootstrap.cs, GameEntryPoint.cs, LevelEntryPoint.cs
DI/         GameLifetimeScope.cs, LevelLifetimeScope.cs, MenuLifetimeScope.cs

Editor/                                     # Game.Editor (editor-only tooling)
  LevelTools/ Level authoring + validation windows
  Validation/ Asset/level validators run in editor + CI
  BuildTools/ Build scripts, version stamping

Tests/
  PlayMode/  Cross-feature integration tests  # Game.Tests.PlayMode

Content/                                     # Authored data assets (data, not code)
Levels/     # Level_01.asset ... (LevelData instances)
Configs/
  Player/   # PlayerMovementConfig.asset
  Traps/    # *TrapConfig.asset
  Game/     # GameConfig.asset

Prefabs/
  Player/   Platforms/ Traps/ UI/ LevelObjects/ VFX/

Scenes/
  Boot/     # Boot.unity (root composition)
  MainMenu/ # MainMenu.unity
  Loading/  # Loading.unity (transitions)
  Game/     # Game.unity (gameplay host)

Settings/                                     # URP assets, Input Actions, quality
Materials/ Shaders/
Addressables/ # Addressable group sources (Levels, Sprites, Audio, UI)

Documentation/ # Architecture, LevelDesign, BuildGuide, QA (Section 33)

```

7.2 What belongs in each major folder

Art/ and **Audio/** hold only source sprites/animation frames and raw clips, grouped by subject. No prefabs, no code, no `.asset` data. (Atlasing and compression: Section 21.)

Code/Core/ is the shared foundation every feature depends on, and the only code besides **App/** that is not a feature. - **Core/Contracts/** (`Game.Core.Contracts`) is the engine-light **API surface**: the interfaces features talk to (`ISaveService`, `IAnalyticsService`, `IAudioService`, `IPlayerInput`, `ISceneLoader`, `IObjectPool<T>`, `IGameStateService`, `ITrap`), the cross-feature **signals**, and the shared **data types** (`SaveData`, `GameState`, the `TrapConfig` base). It contains *no* concrete behaviour. Features depend on this, not on each other. - **Core/Runtime/** (`Game.Core`) holds the concrete shared services (save,

analytics, ads, audio, input, scene loading, pooling, the signal bus, the game-state service) plus the DI scaffolding (`RootLifetimeScope` , `IFeatureInstaller`) and common utilities/base classes.

`Code/Features/<Feature>/` is one folder per vertical slice, each a single assembly that owns everything for that feature: its `Runtime/` (services + thin `MonoBehaviour` views), `Config/` (its `ScriptableObject` configs), `DI/` (its `*Installer`), and co-located `Tests/`. The cardinal rule: a feature folder is **self-contained** — you could delete the folder and remove one installer line in `Game.App` , and the rest of the game would still compile (Section 9).

`Code/App/` (`Game.App`) is the composition root. `Bootstrap/` holds the entry points; `DI/` holds the `LifetimeScope`s that register Core services and install every feature. This is the **only** assembly that references all features (Section 8).

`Code/Editor/` holds editor-only tooling (validators, build scripts). It is stripped from player builds and never holds gameplay logic.

`Code/Tests/PlayMode/` holds cross-feature integration tests. Fast, engine-free unit tests live **inside each feature** under `Features/<X>/Tests/` (Section 23). Production code never references test assemblies.

`Content/` holds authored data instances (level assets, tuning configs). Data only, never logic beyond trivial `OnValidate` (Section 31).

`Prefabs/` , `Scenes/` , `Settings/` , `Materials/` , `Shaders/` , `Addressables/` hold composed `GameObjects`; the four scenes (levels are loaded into `Game` , not separate scenes — Sections 12, 16); URP/input/quality assets; the few materials and custom shaders; and streaming content groups.

`Documentation/` holds the living in-repo docs (Section 33).

7.3 Sample file hierarchies with real file names

These concrete examples show how a couple of feature slices and the shared Core fill in — the canonical naming the team should mirror (Section 31). Note how each feature carries its own runtime, config, DI, and tests in one place.

Features/Player — a complete vertical slice (assembly `Game.Features.Player`)

```
Assets/_Project/Code/Features/Player/
  Runtime/
    PlayerView.cs           # MonoBehaviour: holds Rigidbody2D, sprite; applies velocity
    PlayerMovementService.cs # Pure logic: movement, coyote time, jump buffer
    PlayerGroundChecker.cs  # Ground/contact detection
    PlayerDeathHandler.cs   # Raises PlayerDiedSignal; triggers VFX/haptics
    PlayerAnimator.cs       # Drives the sprite animator from movement state
  Config/
    PlayerMovementConfig.cs # ScriptableObject: speed, jump, coyote, buffer values
  DI/
    PlayerInstaller.cs      # Registers this feature's services/views
  Tests/
    PlayerMovementTests.cs  # EditMode tests (assembly Game.Features.Player.Tests)
```

Features/Traps — a complete vertical slice (assembly `Game.Features.Traps`)

```
Assets/_Project/Code/Features/Traps/
  Runtime/
    TrapBase.cs             # Abstract MonoBehaviour implementing ITrap (Core.Contracts)
    TrapTrigger.cs          # Trigger zone that activates a referenced trap
```



```

FallingPlatformTrap.cs    # Platform that drops after a delay
SpikeTrap.cs FakeFloorTrap.cs FakeDoorTrap.cs ReverseControlsTrap.cs
Config/
FallingPlatformTrapConfig.cs # : TrapConfig (Core.Contracts) – designer tuning
SpikeTrapConfig.cs ...
DI/
TrapsInstaller.cs        # Registers this feature's trap factory/services
Tests/
TrapStateTests.cs        # EditMode tests (assembly Game.Features.Traps.Tests)

```

Core — the shared foundation (assemblies `Game.Core.Contracts` + `Game.Core`)

```

Assets/_Project/Code/Core/Contracts/          # Game.Core.Contracts (no concretes, engine-
light)
Save/ISaveService.cs      Save/SaveData.cs
Audio/IAudioService.cs    Input/IPlayerInput.cs
Analytics/IAalyticsService.cs Ads/IAdsService.cs
Scenes/ISceneLoader.cs    Pooling/IObjectPool.cs
State/IGameStateService.cs State/GameState.cs
Traps/ITrap.cs            Traps/TrapConfig.cs
Events/ISignalBus.cs      Events/Signals/PlayerDiedSignal.cs ...

Assets/_Project/Code/Core/Runtime/            # Game.Core (concrete shared services + DI)
DI/RootLifetimeScope.cs  DI/IFeatureInstaller.cs
Save/JsonSaveService.cs  Save/SaveMigrationService.cs
Audio/AudioService.cs    Input/MobilePlayerInput.cs
Scenes/UnitySceneLoader.cs Events/SignalBus.cs
Pooling/ObjectPool.cs    Pooling/EffectPoolService.cs
State/GameStateService.cs State/States/BootState.cs ...

```

App — the single composition root (assembly `Game.App`)

```

Assets/_Project/Code/App/
Bootstrap/
GameBootstrap.cs        # First object in the Boot scene
GameEntryPoint.cs       # App startup: warm save, enter first state
LevelEntryPoint.cs      # Per-level startup/tick
DI/
GameLifetimeScope.cs    # Root scope: Core services + installs all features
LevelLifetimeScope.cs   # Child scope: installs per-level features
MenuLifetimeScope.cs    # Menu scope: installs the UI feature for the menu

```

Why the shared API surface lives in `Core.Contracts` and concretes elsewhere.

`ISaveService` is a *shared contract* the whole game depends on; `JsonSaveService` is one implementation. Keeping the interface (and shared data/signals) in `Game.Core.Contracts` and the concrete in `Game.Core` means every feature compiles and tests against the contract, the JSON/file details (or a future cloud-save) can change without touching a caller, and — crucially — **features never need to reference each other** to collaborate. They share Core's contracts and communicate through Core's signal bus (Section 9).

8. Assembly Definition Strategy

Assembly Definitions (`.asmdef`) split the codebase into separately compiled assemblies. In a feature-first project they do double duty: they keep iteration fast (editing one feature recompiles only that feature and `App`), and they **enforce feature isolation through the compiler** — if `Game.Features.Player` does

not reference `Game.Features.Traps`, then a Player file reaching into a Traps type is a build error, not a code-review nit. The result is a set of vertical slices that can be developed, tested, and reasoned about independently.

8.1 The assemblies

One assembly for the shared contracts, one for shared concretes, one per feature (each with a co-located test assembly), one composition-root assembly, plus editor and integration-test assemblies.

Assembly	Folder	UnityEngine	VContainer
<code>Game.Core.Contracts</code>	<code>Code/Core/Contracts</code>	No (avoid)	No
<code>Game.Core</code>	<code>Code/Core/Runtime</code>	When needed	Allowed
<code>Game.Features.Player</code>	<code>Code/Features/Player</code>	Yes	Allowed
<code>Game.Features.Traps</code>	<code>Code/Features/Traps</code>	Yes	Allowed
<code>Game.Features.Levels</code>	<code>Code/Features/Levels</code>	Yes	Allowed
<code>Game.Features.Progression</code>	<code>Code/Features/Progression</code>	No (prefer)	Allowed
<code>Game.Features.Ui</code>	<code>Code/Features/Ui</code>	Yes	Allowed
<code>Game.App</code>	<code>Code/App</code>	Yes	Yes
<code>Game.Editor</code>	<code>Code/Editor</code>	Yes (Editor)	As needed
<code>Game.Features.<X>.Tests</code>	<code>Code/Features/<X>/Tests</code>	Per feature	As needed
<code>Game.Tests.PlayMode</code>	<code>Code/Tests/PlayMode</code>	Yes	As needed

Each feature ships its own `Game.Features.<X>.Tests` EditMode assembly co-located in the feature folder, so a feature's tests travel with the feature (Section 23).

8.2 Per-assembly contents and references

Game.Core.Contracts is the shared **API surface**: service interfaces (`ISaveService`, `IAnalyticsService`, `IAudioService`, `IPlayerInput`, `ISceneLoader`, `IObjectPool<T>`, `IGameStateService`, `ITrap`), cross-feature **signals**, and shared **data types** (`SaveData`, `GameState`, the `TrapConfig` base). **References**: none. **UnityEngine**: avoided — keeping the contract surface engine-light keeps the testable core honest and lets pure logic compile in headless CI (Section 23).

Game.Core contains the concrete cross-cutting services that fulfil those contracts — `JsonSaveService`, analytics/ads adapters, `AudioService`, the input adapter, `UnitySceneLoader`, pooling (`ObjectPool`, `EffectPoolService`), the `SignalBus`, the `GameStateService` — plus the DI scaffolding (`RootLifetimeScope`, `IFeatureInstaller`) and common utilities/base classes. **References**: `Game.Core.Contracts`. **UnityEngine**: allowed where a service needs it (e.g. `Application.persistentDataPath`, `AudioSource`).

Game.Features.<X> is one self-contained vertical slice. It owns its runtime services, thin `MonoBehaviour` views, ScriptableObject configs, and its `*Installer`. **References**: `Game.Core.Contracts` (always) and `Game.Core` (for base classes/utilities, usually). **It must NOT reference any other `Game.Features.*` assembly.** When a feature needs something another feature provides, it depends on a Core contract or

reacts to a Core signal — never on the other feature directly. **UnityEngine:** yes (except a logic-only feature like `Progression`, which can stay engine-free and therefore trivially testable).

Game.App contains the VContainer `LifetimeScope`s and entry points — the composition root. It is the **only** assembly that references every feature, because composition is where the slices are assembled into a game. **References:** `Game.Core.Contracts`, `Game.Core`, and all `Game.Features.*`. **VContainer:** yes.

Game.Editor contains editor tooling and build scripts. **References:** selected runtime assemblies it inspects/validates. **Editor-only:** yes; never in a player build.

Game.Features.<X>.Tests are fast EditMode tests for one feature. **References:** that feature plus `Game.Core.Contracts` (and `Game.Core` for fakes). **Game.Tests.PlayMode** holds scene/integration tests and **references** `Game.App` and the features it exercises.

8.3 Reference rules (the contract)

Assembly reference rules — enforced by `.asmdef`

```
Game.Core.Contracts
  References: none
  UnityEngine: avoided (engine-light API surface)

Game.Core
  References: Game.Core.Contracts
  UnityEngine: allowed where needed

Game.Features.Player | .Traps | .Levels | .Ui
  References: Game.Core.Contracts, Game.Core
  Must NOT reference any other Game.Features.*
  UnityEngine: allowed

Game.Features.Progression
  References: Game.Core.Contracts (Game.Core if needed)
  UnityEngine: avoided (logic-only, fully unit-testable)

Game.App (composition root — the only omniscient assembly)
  References: Game.Core.Contracts, Game.Core, ALL Game.Features.*
  VContainer: yes

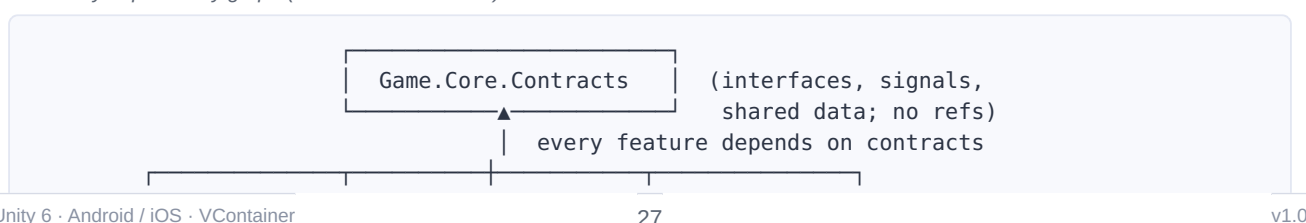
Game.Editor
  References: selected runtime assemblies
  Editor-only: yes

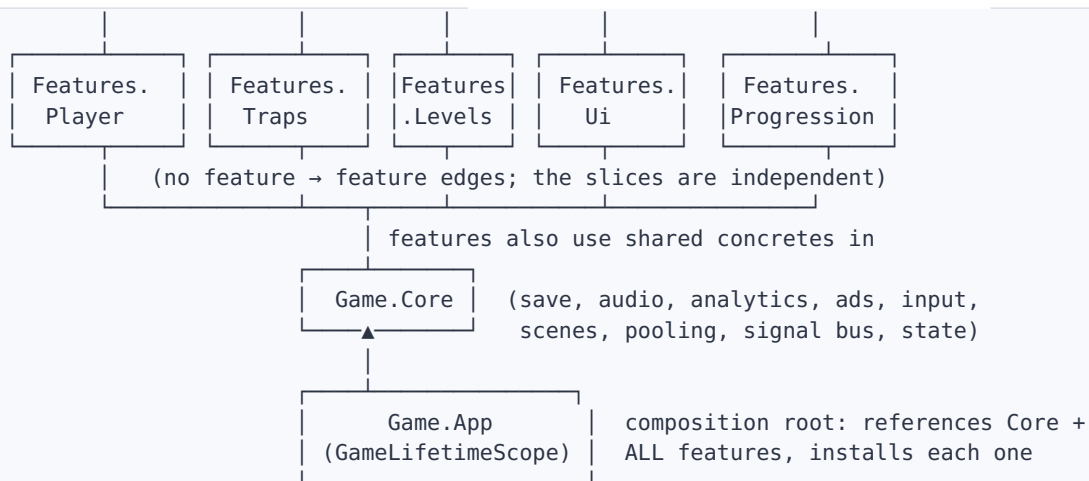
Game.Features.<X>.Tests
  References: Game.Features.<X>, Game.Core.Contracts (, Game.Core)

Game.Tests.PlayMode
  References: Game.App + exercised features
```

8.4 Dependency diagram

Assembly dependency graph (arrows = "references")





Editor: `Game.Editor` → (selected runtime assemblies)
 Tests: `Game.Features.<X>.Tests` → `Game.Features.<X>`, `Game.Core.Contracts`
`Game.Tests.PlayMode` → `Game.App` + exercised features

8.5 Why this pays off

- **Compile time.** Editing a trap recompiles `Game.Features.Traps` and `Game.App`, not the player, levels, UI, or progression slices. On a large project this is the difference between a 2-second and a 30-second iteration.
- **Feature isolation is mechanical.** A Player file *cannot* reach into a Traps type, because `Game.Features.Player` does not reference `Game.Features.Traps`. The compiler enforces the slice boundary; cross-feature collaboration is funnelled through Core contracts and signals (Section 9).
- **No circular references.** Unity forbids assembly cycles outright, and the star-shaped graph (features → Core, App → everything) has none by construction. Two features that seem to need each other instead share a contract or a signal in Core.
- **Independent testing.** Each feature's EditMode tests reference only that feature and Core contracts, so they run headless in CI in milliseconds, and a logic-only feature like `Progression` is engine-free and trivially testable (Sections 23, 26).
- **Delete-ability.** A correctly bounded feature can be removed by deleting its folder and one installer line in `Game.App` — the strongest possible proof that the slice was truly self-contained.

Placement of VContainer. Only the assemblies that compose or consume injection reference VContainer: `Game.App` (the composition root) and the feature assemblies that use `[Inject]` / constructor injection in their installers and services. Keep `Game.Core.Contracts` free of VContainer entirely — contracts are plain interfaces and data, resolved by the container but never aware of it.

Part IV · Code Architecture

With the project skeleton and assembly boundaries fixed (Part III), the code architecture becomes a set of rules about *which feature may know about what*. This part defines the feature-first structure (a shared Core plus independent slices), shows how VContainer wires it at runtime, catalogues the

design patterns the team will actually use, and specifies the scene and game-state flow that the whole game runs through.

9. Industry-Level Code Architecture

The architecture is **feature-first** (also called *vertical-slice* or *feature-folder* architecture): code is organised around the things the player experiences — Player, Traps, Levels, Progression, UI — and each of those is a self-contained slice that owns its logic, its views, its data, and its wiring. Underneath sits a thin **Core**: the shared contracts and cross-cutting services every slice builds on. A single **App** assembly composes the slices into a running game. The aim is not architectural purity for its own sake; it is **independent, deletable features, replaceable infrastructure, and the ability to add 100+ levels and new trap types without the codebase rotting**. Each slice is an assembly (Section 8) so the boundaries are compiler-enforced.

9.1 The shape: Core, Features, App

Core — the shared foundation. Two assemblies. `Game.Core.Contracts` is the engine-light **API surface**: the interfaces features talk to (`ISaveService`, `IAudioService`, `IPlayerInput`, `IAnalyticsService`, `ISceneLoader`, `IObjectPool<T>`, `IGameStateService`, `ITrap`), the cross-feature **signals** (`PlayerDiedSignal`, `LevelCompletedSignal`...), and the shared **data types** (`SaveData`, `GameState`, the `TrapConfig` base). `Game.Core` provides the concrete cross-cutting **services** that fulfil those contracts (save, audio, analytics, ads, input, scene loading, pooling, the signal bus, the game-state service) plus the DI scaffolding and common utilities. Core knows about *no* feature.

Features — the vertical slices. One assembly each: `Player`, `Traps`, `Levels`, `Progression`, `Ui`. A slice contains everything for that capability end-to-end — its runtime services (pure logic), its thin `MonoBehaviour` views, its ScriptableObject configs, its DI installer, and its tests. A slice depends only on Core; it never depends on another slice.

App — the composition root. `Game.App` is the only assembly that references every feature. Its `LifetimeScope`s register Core's services and install each feature's installer, assembling the slices into a game (Section 10).

9.2 The two rules that hold it together

Feature-first architecture lives or dies by two dependency rules, both enforced by assembly definitions (Section 8):

1. **Features depend on Core, never on each other.** `Game.Features.Player` may reference `Game.Core.Contracts` and `Game.Core`, but **not** `Game.Features.Traps`. This is what keeps a slice independently buildable, testable, and deletable.
2. **Core never depends on a feature.** Contracts and shared services know nothing about Player, Traps, or UI. Dependencies point *toward* Core (and toward `Core.Contracts` within Core), and `App` sits on top referencing everything.

When two features must collaborate, they do it through Core, two ways: a **shared contract** (the providing feature implements a `Core.Contracts` interface that the consuming feature resolves via DI) or a **signal** (one feature publishes a `Core.Contracts` signal on the `ISignalBus`; any feature can

subscribe). Player publishing `PlayerDiedSignal` while UI, Progression, and the audio service react — with no direct reference between them — is the canonical example.

9.3 Who may reference whom

Assembly	May reference	May NOT reference	UnityEngine
<code>Game.Core.Contracts</code>	(nothing)	everything else	No (avoid)
<code>Game.Core</code>	<code>Core.Contracts</code>	any feature, <code>App</code>	When needed
<code>Game.Features.<X></code>	<code>Core.Contracts</code> , <code>Core</code>	any other feature, <code>App</code>	Yes*
<code>Game.App</code>	<code>Core.Contracts</code> , <code>Core</code> , all features	—	Yes

* `Game.Features.Progression` is logic-only and stays engine-free so it is trivially unit-testable.

The single most important rule: **features are independent slices that meet only at Core**. A feature is wired to Core's concrete services by `VContainer` at the composition root (Section 10), never by direct construction or by reaching into a sibling feature.

9.4 Architecture diagram

Feature-first dependency flow



9.5 Keeping the design honest

- **Why a slice must be deletable.** The acid test of a feature boundary is: can you delete the feature's folder and one installer line in `Game.App` and still compile? If removing `Features.Traps` breaks `Features.Player`, the slice leaked — Player reached into Traps instead of going through a Core contract or signal. The `asmdef` makes that leak a build error before it ever ships (Section 8).
- **Why cross-feature talk goes through Core.** Direct feature-to-feature calls re-introduce the tangle feature architecture exists to prevent: a change in Traps would ripple into Player, UI, and Levels. Routing collaboration through a `Core.Contracts` interface or an `ISignalBus` signal means each slice only ever depends on the stable shared surface, not on its neighbours' internals.
- **Why Core.Contracts stays engine-light.** The shared interfaces, signals, and data types are the seam everything compiles against. Keeping `UnityEngine` out of them keeps the contract surface portable and the pure-logic features (and Core's logic) unit-testable in headless CI in milliseconds.

(Section 23). The moment a `Vector3` leaks into a contract, that guarantee weakens — so the `asmdef` discourages it.

- **How features stay testable.** A slice's logic (movement math, trap state, progression rules) lives in plain C# services injected into thin `MonoBehaviour` views. The service has no `Update`; the view (or an entry point) calls `service.Tick(dt)`. The service is unit-tested with fakes for the Core contracts it needs — no scene, no singletons, no sibling features (Section 23).
- **How circular dependencies are avoided.** They cannot exist: Unity forbids assembly cycles (Section 8), and the star-shaped feature graph has none. If two features appear to need each other, that is the signal to introduce a shared contract or signal in Core and break the dependency.

10. Dependency Injection with VContainer

VContainer is the composition mechanism for the entire game. It replaces static singletons and `FindObjectOfType` lookups with explicit, constructor-injected dependencies wired at well-defined composition roots (the `LifetimeScope`s in `Game.App`). In a feature-first project it plays a second role: each feature contributes a small **installer** that registers only that feature's services, and the root scope simply **installs every feature**. This is what makes the slice boundaries of Section 9 practical instead of aspirational — a feature owns its own wiring.

10.1 Why VContainer

- **Testability.** A service whose dependencies arrive through its constructor can be instantiated in a test with fakes. No scene, no singletons, no sibling features.
- **Fewer static managers.** Instead of `GameManager.Instance.Save`, a class declares `ISaveService` in its constructor and receives the one registered implementation. Lifetimes are explicit, not "alive forever because it's static."
- **Feature isolation.** Each feature's installer registers its own services against Core contracts. A feature never `new`s another feature's types; it asks for a Core contract and the container supplies whatever implementation was installed.
- **Dependency visibility.** A class's constructor is an honest list of what it needs. Hidden dependencies (static calls, scene singletons) disappear.
- **Scene and level composition.** Each scene/level gets a `LifetimeScope`; child scopes inherit parent registrations, so per-level objects can depend on app-wide Core services without globals.

10.2 Scopes and feature installers

Two ideas combine: **scopes** (lifetime/teardown boundaries that mirror the scene flow, Section 12) and **feature installers** (each feature's self-contained registration).

- **Root scope** (`GameLifetimeScope`, in the Boot scene) registers Core's app-lifetime services — save, audio, analytics, input, scene loading, pooling, the signal bus, the game-state service — and installs the always-on features (e.g. `Progression`). App-lifetime → **Singleton**.
- **Menu scope** (`MenuLifetimeScope`) installs the UI feature configured for the menu.
- **Level scope** (`LevelLifetimeScope`, created per level) installs the per-level features — `Levels`, `Traps`, `Player`, and the in-game `Ui` — as **Scoped**, so every level gets fresh instances that are disposed on restart/teardown (clean and leak-free, Section 12).

Each feature exposes an **installer** implementing a tiny `IFeatureInstaller` contract (defined in `Game.Core`). A scope installs a feature by handing the builder to that installer — the scope never needs to know the feature's internal types.

```
Assets/_Project/Code/Core/Runtime/DI/IFeatureInstaller.cs · asmdef: Game.Core

using VContainer;

namespace Game.Core.DI
{
    /// A feature's self-contained registration. The App's LifetimeScopes
    /// call Install; no scope needs to know a feature's internal types.
    public interface IFeatureInstaller
    {
        void Install(IContainerBuilder builder);
    }
}
```

Singleton vs. Scoped. Register Core's app-lifetime services (`ISaveService` , `IAudioService` , `IAalyticsService` , `IGameStateService` , `ISignalBus`) as **Singleton** in the root scope. Register per-level feature services (`PlayerMovementService` , `LevelRuntime` , the trap services) as **Scoped** in the level scope so each level gets a fresh instance disposed on restart/teardown — this is what makes restart clean and leak-free (Section 12).

10.3 A feature installer

Each feature registers *only its own* types, against Core contracts. Here is the Player slice's installer — it lives inside `Game.Features.Player` , not in `App` , so the feature owns its wiring.

```
Assets/_Project/Code/Features/Player/DI/PlayerInstaller.cs · asmdef: Game.Features.Player

using VContainer;
using Game.Core.DI;

namespace Game.Features.Player
{
    public sealed class PlayerInstaller : IFeatureInstaller
    {
        private readonly PlayerMovementConfig _config;
        private readonly PlayerView _view;

        public PlayerInstaller(PlayerMovementConfig config, PlayerView view)
        {
            _config = config;
            _view = view;
        }

        public void Install(IContainerBuilder builder)
        {
            builder.RegisterInstance(_config);
            builder.RegisterInstance(_view);
            builder.Register<PlayerMovementService>(Lifetime.Scoped);
            // IPlayerInput is a Core contract; its implementation is
            // registered by the root scope and resolved up the parent chain.
        }
    }
}
```



```

    }
}

```

10.4 The root composition root

The root scope registers Core services as singletons and installs the always-on features. Note there are no `static` references anywhere — every dependency is registered once and resolved by constructor.

Assets/_Project/Code/App/DI/GameLifetimeScope.cs · asmdef: Game.App

```

using UnityEngine;
using VContainer;
using VContainer.Unity;
using Game.Core.Contracts.Save;
using Game.Core.Contracts.Audio;
using Game.Core.Contracts.Analytics;
using Game.Core.Contracts.Input;
using Game.Core.Contracts.Scenes;
using Game.Core.Contracts.State;
using Game.Core.Contracts.Events;
using Game.Features.Progression;

namespace Game.App.DI
{
    public sealed class GameLifetimeScope : LifetimeScope
    {
        [SerializeField] private GameConfig _gameConfig;

        protected override void Configure(IContainerBuilder builder)
        {
            builder.RegisterInstance(_gameConfig);

            // Core cross-cutting services (app-lifetime singletons).
            builder.Register<ISignalBus, SignalBus>(Lifetime.Singleton);
            builder.Register<ISaveService, JsonSaveService>(Lifetime.Singleton);
            builder.Register<IAudioService, AudioService>(Lifetime.Singleton);
            builder.Register<IPlayerInput, MobilePlayerInput>(Lifetime.Singleton);
            builder.Register<ISceneLoader, UnitySceneLoader>(Lifetime.Singleton);
            builder.Register<IAnalyticsService, AnalyticsService>(Lifetime.Singleton);
            builder.Register<IGameStateService, GameStateService>(Lifetime.Singleton);

            // Always-on features install themselves.
            new ProgressionInstaller().Install(builder);

            builder.RegisterEntryPoint<GameEntryPoint>();
        }
    }
}

```

What each line does. `RegisterInstance(_gameConfig)` exposes a serialized config as a resolvable dependency. Each `Register<TInterface, TImpl>(Lifetime.Singleton)` binds a `Core.Contracts` interface to its `Game.Core` implementation for the app's lifetime. `new ProgressionInstaller().Install(builder)` lets the always-on `Progression` feature register its own services — the scope never touches `Progression`'s internal types. `RegisterEntryPoint<GameEntryPoint>()` registers a startup object whose `Start/Tick` `VContainer` drives through the player loop (Section 10.6).

10.5 The level scope and entry points

Each level scope is a **child** of the root scope. It installs the per-level feature slices; its serialized references are the *view* objects authored in the level prefab.

```
Assets/_Project/Code/App/DI/LevelLifetimeScope.cs · asmdef: Game.App

using UnityEngine;
using VContainer;
using VContainer.Unity;
using Game.Features.Player;
using Game.Features.Traps;
using Game.Features.Levels;
using Game.Features.Ui;

namespace Game.App.DI
{
    public sealed class LevelLifetimeScope : LifetimeScope
    {
        [SerializeField] private LevelData _levelData;
        [SerializeField] private PlayerView _playerView;
        [SerializeField] private PlayerMovementConfig _playerConfig;
        [SerializeField] private HudView _hudView;

        protected override void Configure(IContainerBuilder builder)
        {
            // Each per-level feature installs its own services (Scoped).
            new LevelsInstaller(_levelData).Install(builder);
            new TrapsInstaller().Install(builder);
            new PlayerInstaller(_playerConfig, _playerView).Install(builder);
            new UiInstaller(_hudView).Install(builder);

            builder.RegisterEntryPoint<LevelEntryPoint>();
        }
    }
}
```

Because this scope is a **child** of the root scope, the per-level services can receive both the views installed here and the app-wide Core services (`IAudioService`, `ISignalBus`, ...) registered in the root — VContainer resolves up the parent chain. When the level unloads, the scope and all its `Scoped` instances are disposed, guaranteeing a clean slate for the next attempt. Crucially, `LevelLifetimeScope` only ever touches each feature's *installer*; it never reaches into a feature's internal types, so the slice boundaries of Section 9 hold even at the composition root.

The entry points are the startup/per-frame hooks VContainer drives. `LevelEntryPoint` (in `Game.App`) initialises the level and ticks the per-level orchestration.

```
Assets/_Project/Code/App/Bootstrap/LevelEntryPoint.cs · asmdef: Game.App

using VContainer.Unity;
using Game.Features.Levels;

namespace Game.App.Bootstrap
{
    public sealed class LevelEntryPoint : IStartable, ITickable
    {
        private readonly LevelRuntime _level;
```

```

    public LevelEntryPoint(LevelRuntime level)
    {
        _level = level;
    }

    public void Start() => _level.Initialize();
    public void Tick() => _level.Tick();
}

```

The root scope's `GameEntryPoint` plays the analogous role at the app level — warming save data and routing into the first state — while a `MenuEntryPoint` wires the menu scope.

Assets/_Project/Code/App/Bootstrap/GameEntryPoint.cs · asmdef: `Game.App`

```

using VContainer.Unity;
using Game.Core.Contracts.State;
using Game.Core.Contracts.Save;

namespace Game.App.Bootstrap
{
    public sealed class GameEntryPoint : IStartable
    {
        private readonly IGameStateService _stateService;
        private readonly ISaveService _saveService;

        public GameEntryPoint(IGameStateService stateService, ISaveService saveService)
        {
            _stateService = stateService;
            _saveService = saveService;
        }

        public void Start()
        {
            _ = _saveService.Load(); // warm the save data
            _stateService.ChangeState<BootState>();
        }
    }
}

```

Assets/_Project/Code/App/DI/MenuLifetimeScope.cs · asmdef: `Game.App`

```

using UnityEngine;
using VContainer;
using VContainer.Unity;
using Game.Features.Ui;

namespace Game.App.DI
{
    public sealed class MenuLifetimeScope : LifetimeScope
    {
        [SerializeField] private MenuView _menuView;

        protected override void Configure(IContainerBuilder builder)
        {
            // The Ui feature installs its menu presenters for this scope.
            new UiInstaller(_menuView).Install(builder);
        }
    }
}

```

```

        builder.RegisterEntryPoint<MenuEntryPoint>();
    }
}
}

```

10.6 IStartable, ITickable, IFixedTickable — and not overusing them

VContainer's player-loop interfaces let plain C# classes participate in Unity's update loop *without being MonoBehaviours*:

- **IStartable.Start()** — one-time initialization after the container is built. Use for level setup, subscribing to events, spawning initial objects.
- **ITickable.Tick()** — once per frame (`Update`). Use for input polling and frame-rate-dependent logic.
- **IFixedTickable.FixedTick()** — once per physics step (`FixedUpdate`). Use for applying forces/velocity to `Rigidbody2D` (player movement, Section 14).

Do not overuse them. Not every service should tick. A `SaveService` reacts to calls; it has no `Tick`. Prefer **one** ticking orchestrator per scope (the `LevelEntryPoint` / `LevelRuntime`) that pumps the few services that truly need per-frame updates, rather than registering dozens of `ITickable`s. Fewer tick entry points means a clearer frame and less overhead (Section 21).

10.7 Injecting into MonoBehaviours and avoiding service-locator misuse

When a `MonoBehaviour` genuinely needs an injected dependency (for example a `PlayerView` that needs the `IAudioService`), use VContainer's `[Inject]` method injection on that component, registered through the scope. **Constructor injection is preferred for all pure C# classes**; method/property injection is the exception reserved for `MonoBehaviour`s, which Unity constructs.

The anti-pattern to avoid is treating the container as a **service locator** — passing `IObjectResolver` around and calling `resolver.Resolve<T>()` deep in the code. That re-hides dependencies the constructor was supposed to make visible, defeating the entire point. Resolve only at composition roots; everywhere else, declare what you need and let it be injected.

11. Recommended Design Patterns

These are the patterns the team will actually use, each tied to a concrete place in *PLUMMET*. The rule of thumb: use the pattern when it removes a real dependency or a real branch, not because it is on a list.

11.1 Pattern catalogue

Dependency Injection — *Why*: removes static coupling, enables testing. *Where*: everywhere, via VContainer scopes (Section 10). *Solves*: hidden/global dependencies. *Misuse*: using the container as a service locator.

State Machine — *Why*: models a system with distinct, mutually exclusive modes and legal transitions. *Where*: the game-state service (Section 13) and the player's local state (idle/run/jump/dead, Section 14). *Solves*: tangled boolean flags (`isDead && !isPaused && isLoading`). *Misuse*: a state machine for two states — an `enum` and a branch is enough.

Factory — *Why*: centralizes creation of families of objects. *Where*: trap creation, level object spawning, VFX. *Solves*: scattered `Instantiate` calls with setup logic. *Misuse*: a factory that only ever makes one type — just construct it.

Strategy — *Why*: swap an algorithm behind an interface. *Where*: trap activation behaviours, difficulty/ramp policies. *Solves*: large `switch` over behaviour type. *Misuse*: a strategy interface with a single implementation that will never grow.

Observer / Event Bus — *Why*: decouple producers from consumers. *Where*: gameplay events (`PlayerDied`, `LevelCompleted`, `TrapTriggered`) that analytics, audio, and UI all react to. *Solves*: a death handler that must know about UI, audio, and analytics. *Misuse*: a global event bus for everything, which hides flow and creates spaghetti — prefer narrowly scoped C# `event`s or typed signals, and reserve a bus for genuinely cross-cutting events.

Command — *Why*: turn a request into an object (queue, log, undo). *Where*: debug/cheat actions (Section 24), input rebinding, replayable level actions. *Solves*: hard-to-test inline side effects. *Misuse*: commands for trivial direct calls.

Repository — *Why*: abstract data access behind a collection-like interface. *Where*: level metadata and progress (`ILevelRepository`, `ISaveService`). *Solves*: save/load details leaking into use cases. *Misuse*: a repository over a single in-memory list with no persistence concern.

Object Pool — *Why*: reuse instances to avoid runtime allocation and GC spikes. *Where*: death VFX, particle bursts, frequently spawned trap effects (Section 21). *Solves*: hitching from `Instantiate`/`Destroy` churn. *Misuse*: pooling rare or singleton objects.

Presenter (MVP) — *Why*: keep UI logic out of views. *Where*: every screen (Section 18). *Solves*: business logic in `MonoBehaviour` UI scripts. *Misuse*: a presenter that is a pass-through with no logic — acceptable, but do not add ceremony where a direct binding suffices.

Composition over inheritance — *Why*: build behaviour by combining small parts rather than deep class trees. *Where*: traps (a trap *has* a trigger, a reset, an effect) and player (view + ground checker + death handler). *Solves*: fragile base classes and "diamond" hierarchies. *Misuse*: none, generally — but do not fragment a cohesive class into a dozen micro-components for its own sake.

ScriptableObject configuration — *Why*: designer-tunable data assets, no recompiling to tweak values. *Where*: `PlayerMovementConfig`, trap configs, level data, audio banks (Sections 14–17). *Solves*: magic numbers hardcoded in scripts. *Misuse*: putting *logic* in ScriptableObjects, or using them as runtime mutable singletons (they persist edits in the editor).

11.2 State pattern interfaces

The game-state machine is defined by two small interfaces, both in `Game.Core.Contracts`. The state itself is a plain C# type; the service that drives transitions is a Core service.

Assets/_Project/Code/Core/Contracts/State/IGameState.cs · asmdef: `Game.Core.Contracts`

```
namespace Game.Core.Contracts.State
{
    public interface IGameState
    {
        void Enter();
        void Exit();
        void Tick();
    }
}
```

```
}
}
```

Assets/_Project/Code/Core/Contracts/State/IGameStateService.cs · asmdef: Game.Core.Contracts

```
namespace Game.Core.Contracts.State
{
    public interface IGameStateService
    {
        void ChangeState<TState>() where TState : IGameState;
    }
}
```

A concrete `GameStateService` (in `Game.Core`) holds the registered states, calls `Exit()` on the current state and `Enter()` on the next, and forwards `Tick()` to the active state each frame. States are resolved through `VContainer` so a state can depend on services (a `LoadingState` needs `ISceneLoader`).

11.3 The game states

The machine models these states (detailed transitions in Section 13):

- **Boot** — load global services and save data, then go to `MainMenu`.
- **MainMenu** — title and navigation.
- **Loading** — cover transitions while a level/scene streams in.
- **LevelStart** — brief level intro/spawn before control is handed over.
- **Playing** — normal gameplay; input live, traps armed.
- **PlayerDead** — death reaction (VFX, haptics) before restart.
- **Restarting** — reset traps and player to level start (fast path).
- **LevelComplete** — success beat, persist progress, unlock next.
- **Paused** — gameplay frozen, pause menu shown.
- **AppSuspended** — app backgrounded; persist and quiet audio.

Modeling these explicitly (rather than as flags) is what keeps "can the player move right now?" answerable by a single question — *which state are we in?* — instead of a fragile conjunction of booleans.

12. Scene Architecture

The game uses **four scenes**, not one-scene-per-level. Levels are prefabs (optionally `Addressable`) loaded into the persistent `Game` scene. This is the key decision behind sub-0.3-second restarts (Section 14, Section 21): restarting reloads a prefab, not a scene.

Scene set

```
Boot.unity
MainMenu.unity
```

Loading.unity
Game.unity

12.1 Scene responsibilities

- **Boot** loads and composes global services (the project/game `LifetimeScope`), reads save data, then transitions out. It is the app's composition root and runs once.
- **MainMenu** hosts the menu UI and the `MenuLifetimeScope`; it handles play, level select, and settings.
- **Loading** is a lightweight transition scene (or an additive overlay) that hides streaming hitches behind a clean wipe.
- **Game** is the gameplay host. It stays loaded across levels; the current level prefab is loaded into it and swapped on completion/restart.

12.2 What persists, what is recreated

- **Persists** (singletons in the root scope): save, audio, analytics, scene loader, game-state service. These survive across scenes via the root `LifetimeScope`, **not** via scattered `DontDestroyOnLoad` calls.
- **Recreated each level** (scoped in the level scope): the player instance, the level's traps, the `PlayerMovementService`, `LevelRuntime`, and the trap services. Recreating them per level guarantees no stale state leaks between attempts.

12.3 How restart works

Restart does **not** reload the `Game` scene. It either (a) resets the existing level objects in place via `TrapResetService` and a player respawn (fastest), or (b) disposes the level scope and re-instantiates the level prefab (clean, still well under the budget). Approach (a) is the default for instant feel; (b) is a fallback for levels whose traps mutate too much world state to reset cheaply (Section 15).

12.4 Avoiding DontDestroyOnLoad abuse

`DontDestroyOnLoad` creates invisible, order-dependent global state — exactly what this architecture avoids. Persistence is instead expressed by **scope lifetime**: objects registered in the root scope live as long as the app; objects in a child scope live as long as that scope. There is no need to mark individual `GameObjects` as undestroyable, and doing so scatters lifetime decisions across the scene hierarchy where they cannot be reasoned about.

12.5 Scene and state flow

Scene + state flow

```

Boot → MainMenu → Loading → Game → LevelComplete → Loading → Game
                        ↑           ↓
                    Restart ← PlayerDeath
  
```

The `IGameStateService` (Section 11) drives these transitions; the `ISceneLoader` implementation performs the actual scene/Addressable loads, so states stay engine-agnostic and testable.

13. Game State Flow

This table is the authoritative specification of every game state: what runs, what input is accepted, what UI is visible, what events can fire, and which states may follow. It is the contract the `GameStateService` (Section 11) and QA (Section 23) both work from.

State	Active systems	Allowed input	Visible UI	Events	Next states
Boot	Service composition, save load	None	Splash	ServicesReady, SaveLoaded	MainMenu
MainMenu	Menu presenters, audio (menu)	UI taps	Title, Play, Level Select, Settings	PlayPressed, LevelSelected	Loading, Settings
Loading	Scene/ Addressable streaming	None (or cancel)	Loading wipe	LoadProgress, LoadComplete	LevelStart, MainMenu
LevelStart	Level scope built, player spawned (frozen)	None	HUD (faded in)	LevelInitialized	Playing
Playing	Movement, traps, camera, audio, analytics	Move, Jump, Pause	HUD, mobile controls	PlayerDied, ReachedDoor, PausePressed	PlayerDead, LevelComplete, Paused, AppSuspended
PlayerDead	Death VFX, haptics, sfx	None	HUD + death flourish	DeathAnimDone	Restarting
Restarting	Trap reset, player respawn	None	HUD	ResetComplete	Playing
LevelComplete	Persist progress, unlock next, sfx	Continue/ tap	Complete banner, Next	ProgressSaved, ContinuePressed	Loading, LevelSelect
Paused	Gameplay frozen, audio ducked	Resume, Restart, Quit	Pause menu	ResumePressed, RestartPressed, QuitPressed	Playing, Restarting, MainMenu
Settings	Settings presenter	UI taps, toggles, sliders	Settings panel	SettingChanged, BackPressed	MainMenu, Paused
AppSuspended	Persist save, mute audio	OS resume	(system)	AppPaused, AppResumed	Paused (or prior state)

Reading the table. Notice that input is gated entirely by state: during `PlayerDead`, `Restarting`, `Loading`, and `LevelStart`, player input is ignored. This is why the game never suffers the classic bug of "I jumped during the death animation and it carried into the respawn" — the input adapter is only polled while the machine is in `Playing` (Section 14). `AppSuspended` is special: it can be entered from almost any state and returns to either `Paused` (gameplay) or the prior menu state on resume.

Part V · Core Gameplay Systems

These three systems — player, traps, levels — are the game. Everything in Parts III–IV exists to keep them small, testable, and cheap to extend. The player model is intentionally simple; the depth comes from the trap framework and the level catalogue that recombines it.

14. Player System

The player is a single silhouette, *Blip*, with a deliberately tight, predictable movement model. Because the challenge is *knowledge* (Section 2), the controls must never feel like the obstacle: movement is responsive, forgiving at the edges (coyote time, jump buffering), and identical on every attempt.

14.1 Responsibilities and split

Player behaviour is split across thin Unity views and a pure-ish service (Section 9):

- `PlayerView` (MonoBehaviour) — owns the `Rigidbody2D` and sprite; exposes methods like `SetVelocity`, `PlayAnim`, and contact data. No decisions.
- `PlayerMovementService` (C#) — reads `IPlayerInput`, applies the movement rules, and tells the view what to do. Fully testable.
- `PlayerGroundChecker` — reports grounded state (via contacts or a short cast on the `Solid / OneWay` layers).
- `PlayerDeathHandler` — on a killbox/trap hit, fires death VFX, haptics, and a respawn request through the state machine (Section 13).

14.2 Movement features

- **Horizontal movement** — velocity-based, constant max speed, near-instant acceleration for crisp control (no floaty ramp).
- **Jump** — impulse on press while grounded or within coyote time; optional variable height by cutting upward velocity on early release.
- **Coyote time** — a short window (~0.1 s) after leaving a ledge during which a jump still registers, so edge jumps feel fair.
- **Jump buffer** — a short window (~0.12 s) before landing during which a jump press is remembered and fires on touchdown.
- **Ground detection** — via `Rigidbody2D` contacts or a thin downward cast; never via a single raycast that can miss at speed.
- **Death / respawn / restart** — death enters `PlayerDead`; respawn places *Blip* at the level's spawn with zeroed velocity; restart resets the whole level (Section 12).

14.3 Input abstraction

The movement service depends on an interface, never on the Input System directly. The interface is the contract; the Input System implementation is a Core service (Section 8).

Assets/_Project/Code/Core/Contracts/Input/IPlayerInput.cs · asmdef: Game.Core.Contracts

```

namespace Game.Core.Contracts.Input
{
    public interface IPlayerInput
    {
        float MoveX { get; }
        bool JumpPressed { get; }
        bool JumpHeld { get; }
    }
}

```

`MobilePlayerInput` (in `Game.Core`) implements this from on-screen buttons (Section 18). A `FakePlayerInput` in the test assembly implements the same interface to drive movement deterministically in PlayMode tests (Section 23). Because the service only sees `IPlayerInput`, the same movement code runs identically under touch, a test harness, or a future gamepad.

14.4 Tuning via ScriptableObject

All feel values live in a designer-editable config asset, never hardcoded (Section 11, Section 31).

Assets/_Project/Code/Features/Player/Config/PlayerMovementConfig.cs · asmdef: Game.Features.Player

```

using UnityEngine;

namespace Game.Features.Player
{
    [CreateAssetMenu(menuName = "Game/Player/Movement Config")]
    public sealed class PlayerMovementConfig : ScriptableObject
    {
        public float MoveSpeed = 6f;
        public float JumpForce = 12f;
        public float CoyoteTime = 0.1f;
        public float JumpBufferTime = 0.12f;
    }
}

```

14.5 Movement service outline

The service is an architectural skeleton — the timing/feel rules are filled in during implementation. The key points are that it is constructor-injected, has no `Update` of its own (it is ticked by an entry point, Section 10), and applies physics through the view/adaptor rather than touching the `Rigidbody2D` from logic code.

Assets/_Project/Code/Features/Player/Runtime/PlayerMovementService.cs · asmdef: Game.Features.Player

```

namespace Game.Features.Player
{
    public sealed class PlayerMovementService
    {
        private readonly IPlayerInput _input;
        private readonly PlayerMovementConfig _config;

        public PlayerMovementService(IPlayerInput input, PlayerMovementConfig config)
        {
            _input = input;
        }
    }
}

```

```

        _config = config;
    }

    public void TickMovement()
    {
        // Read input.
        // Apply horizontal movement.
        // Handle jump buffer.
        // Handle coyote time.
        // Apply movement to PlayerView or Rigidbody2D through an adapter.
    }
}

```

This is an **architectural outline**, not the final implementation. During the Prototype/MVP phases (Section 28) it is expanded with the actual coyote/buffer timers, variable-jump cut, and `FixedTick` application of velocity. Keep the public surface (`TickMovement`) stable so the entry point and tests do not churn.

14.6 Animation states and tuning

Blip uses a tiny animation set (Section 19): 1-frame idle, 2–4-frame walk, 1-frame jump, 1-frame hurt/death. The `PlayerView` drives these from the player's local state machine (idle/run/jump/dead) — animation is presentation only and never gates gameplay. Starting tuning values: `MoveSpeed` ≈ 6 , `JumpForce` ≈ 12 , `CoyoteTime` ≈ 0.1 s, `JumpBufferTime` ≈ 0.12 s, gravity scale tuned for a snappy apex; all final values are set by feel during the vertical slice.

15. Trap System Architecture

The trap system is the content engine of *PLUMMET*. The design goal is that designers compose levels from a **catalogue of reusable, data-driven traps** — never by writing a one-off script per level. A new level is an arrangement of existing trap prefabs with tuned parameters; only a genuinely new *mechanic* requires new code.

15.1 The original trap catalogue

The launch catalogue (all original mechanics):

- **Fake floor** — looks solid, vanishes on contact.
- **Falling platform** — drops shortly after being stepped on.
- **Hidden spike** — invisible until the player is committed.
- **Spike popup** — erupts from the floor on a trigger.
- **Moving spike** — patrols a path.
- **Fake door** — the "exit" that does nothing or kills.
- **Door that moves away** — slides off as the player approaches.
- **Door that kills** — entering it is the trap.
- **Reverse controls** — inverts horizontal input for a window.
- **Invisible trigger** — an unseen zone that arms another trap.

- **Ceiling spike drop** — spikes fall from above on a trigger.
- **Disappearing platform** — blinks out on a timer once entered.
- **Platform that becomes a spike** — morphs underfoot.
- **Fake victory** — a false "level complete" that drops the player.
- **Delayed trap** — fires a beat after it logically "should."
- **Camera trick** — pans/zooms to hide a hazard or mislead.
- **Falling ceiling block** — a heavy block drops on a trigger.
- **Moving wall** — closes a path or shoves the player.
- **Bounce trap** — launches the player unexpectedly.
- **Second-attempt trap** — only activates on the player's *second* try, punishing learned confidence.

15.2 How traps work

Every trap shares a small lifecycle: **armed** → **triggered** → **activated** → **reset**. A `TrapTrigger` (a trigger collider zone) detects the player and calls `Activate()` on its referenced trap. On level restart, `TrapResetService` calls `ResetTrap()` on every trap so the next attempt is identical. Trap parameters (delays, distances, speeds) come from serialized fields or a trap config asset (Section 11) — never hardcoded per level. Traps raise events (`TrapTriggered` with the trap's id) that analytics (Section 22) and audio (Section 17) consume, which is how the team later detects which traps are unfair (too-high death-to-completion ratio).

15.3 Trap interface (Core.Contracts)

The abstraction lives in `Game.Core.Contracts` — it has no engine dependency, so trap *logic* and reset rules are testable without a scene.

Assets/_Project/Code/Core/Contracts/Traps/ITrap.cs · asmdef: `Game.Core.Contracts`

```
namespace Game.Core.Contracts.Traps
{
    public interface ITrap
    {
        bool IsActivated { get; }
        void Activate();
        void ResetTrap();
    }
}
```

15.4 Trap base class (Features.Traps)

`TrapBase` is the abstract `MonoBehaviour` all traps inherit. It enforces the once-only activation and the reset contract; concrete traps implement only `OnActivated` and `OnReset`. This is composition-friendly: a trap *has* a trigger and *has* an effect, rather than a deep inheritance tree.

Assets/_Project/Code/Features/Traps/Runtime/TrapBase.cs · asmdef: `Game.Features.Traps`

```
using UnityEngine;
using Game.Core.Contracts.Traps;

namespace Game.Features.Traps
{

```

```

public abstract class TrapBase : MonoBehaviour, ITrap
{
    public bool IsActivated { get; private set; }

    public void Activate()
    {
        if (IsActivated)
            return;

        IsActivated = true;
        OnActivated();
    }

    public void ResetTrap()
    {
        IsActivated = false;
        OnReset();
    }

    protected abstract void OnActivated();
    protected abstract void OnReset();
}

```

15.5 Example: falling platform trap

A concrete trap. It caches its start transform, flips its `Rigidbody2D` to dynamic after a delay on activation, and fully restores itself on reset — the reset path is what makes restart deterministic.

Assets/_Project/Code/Features/Traps/Runtime/FallingPlatformTrap.cs · asmdef: Game.Features.Traps

```

using UnityEngine;

namespace Game.Features.Traps
{
    public sealed class FallingPlatformTrap : TrapBase
    {
        [SerializeField] private Rigidbody2D _rigidbody;
        [SerializeField] private float _delay = 0.25f;

        private Vector3 _startPosition;

        private void Awake()
        {
            _startPosition = transform.position;
        }

        protected override void OnActivated()
        {
            Invoke(nameof(Fall), _delay);
        }

        private void Fall()
        {
            _rigidbody.bodyType = RigidbodyType2D.Dynamic;
        }

        protected override void OnReset()
        {

```

```

        CancelInvoke();

        transform.position = _startPosition;
        _rigidbody.bodyType = RigidbodyType2D.Kinematic;
        _rigidbody.linearVelocity = Vector2.zero;
        _rigidbody.angularVelocity = 0f;
    }
}
}

```

Production note on timing. `Invoke(nameof(Fall), _delay)` is fine for a prototype, but it is reflection-based and hard to test. In production, replace it with an injected timer service, a coroutine runner, or an `async / UniTask` delay that respects pause and time-scale (Section 13) and can be driven deterministically in tests. The architecture already supports this: inject an `ITimeProvider` /timer through the level scope rather than calling `Invoke` directly.

15.6 Avoiding hardcoded one-off level scripts

The discipline that keeps the trap system maintainable: **a level is data, not code**. If a designer wants "a spike that pops after 0.3 s when the player passes x," that is the existing `SpikeTrap` + `TrapTrigger` with a `0.3` delay — not a new `Level17SpikeScript`. New code is justified only for a mechanic the catalogue genuinely cannot express; when that happens, add it as a new reusable `TrapBase` subclass with serialized parameters so the *next* level gets it for free. This is what makes the 100+ level tail (Section 5, Section 28) cheap.

16. Level Design Pipeline

Levels are where deception meets fairness. A *PLUMMET* level must look trivial, hide exactly one (or a small combination) of tricks, kill the first-time player in a way that is funny, and be perfectly solvable on every attempt after the trick is known. This section defines the concrete rules and a 50-level progression.

16.1 Construction rules

- **Level size.** Single screen by default; short side-scroll only when a trick needs offscreen space. A solved level is 5–20 seconds.
- **Tile / grid.** Author on a consistent grid at 16 PPU (Section 6); platforms and hazards snap to it so spacing reads clearly.
- **Platform spacing & jump distance.** Every required jump must be inside the player's proven jump arc (derived from `PlayerMovementConfig`, Section 14). Never require a jump the movement model cannot make — difficulty comes from *deception*, not from impossible execution.
- **Trap spacing.** One surprise per "beat." Stacking three unrelated traps in two tiles reads as unfair noise; space them so each death has one clear cause.
- **Difficulty curve.** Ramp gradually (see table); introduce one new mechanic at a time, let it breathe for a few levels, then combine it with earlier ones.
- **Fairness rules.** After the first death, the information needed to survive must be visible or learnable. No purely random kills, no off-screen instant death without a tell on the retry.

- **Surprise rules.** The *first* encounter hides information; that is the whole genre. The contract is "surprising once, fair forever after."
- **Onboarding.** The first several levels teach core verbs (move, jump, door) and the *idea* that the game lies, using gentle, non-punishing tricks.
- **Trap readability (in hindsight).** After dying, the player should think "oh, of course," not "how was I supposed to know?" The minimalist art (Section 19) exists to make the trap legible on the second look.
- **Death-reason clarity.** Each death has exactly one obvious cause; this is also logged for analytics (Section 22) to catch ambiguous levels.
- **Naming convention.** `Lvl_{world}_{index}_{shortName}` (for example `Lvl_01_03_FakeFloor`) for prefabs/Addressables, so content is sortable and greppable (Section 31).
- **Validation tools.** An editor validator (Section 24) checks every level for: a spawn point, a reachable door, all traps wired to a trigger and resettable, and no required jump beyond the movement arc — run in-editor and in CI (Section 26).

16.2 Fast-restart design

Levels are authored so the *whole* level resets cheaply (Section 12): traps restore via `OnReset`, no trap permanently destroys level geometry it cannot rebuild, and the spawn is a fixed point. If a trick mutates the world irreversibly, the level uses the prefab-reinstantiate restart path instead — but designers are nudged toward resettable tricks to keep restart at the fastest tier.

16.3 Fifty-level progression plan

The first 50 levels across the opening worlds. Difficulty is rated 1 (gentle) to 5 (cruel-but-fair).

Level range	New mechanic	Player lesson	Trap examples	Difficulty
1–3	Move, jump, reach door	The basic verbs; the door is the goal	(none — clean tutorial)	1
4–6	The game lies	Not everything is what it seems	Fake floor, hidden spike	1–2
7–9	Falling platforms	Standing still can kill you	Falling platform, delayed drop	2
10–12	Popup & ceiling spikes	Hazards come from any direction	Spike popup, ceiling spike drop	2
13–15	Deceptive doors	The exit itself can betray you	Fake door, door that moves away	2–3
16–18	Disappearing platforms	Footing is temporary	Disappearing platform, blink-out	3
19–21	Moving hazards	Time your movement to patterns	Moving spike, moving wall	3
22–24	Reverse controls	Your own inputs can be flipped	Reverse controls + spikes	3
25–27	Camera tricks	Trust the layout, not the framing	Camera trick hiding a pit	3–4
28–30	Morphing terrain	Platforms aren't permanent objects	Platform → spike, fake victory	4

Level range	New mechanic	Player lesson	Trap examples	Difficulty
31–33	Bounce & launch	Momentum can be weaponized	Bounce trap into hazard	4
34–36	Falling ceiling blocks	Overhead threats on triggers	Falling block, invisible trigger	4
37–40	Combinations I	Two mechanics at once	Falling platform + popup spike	4
41–44	Second-attempt traps	The retry itself is a trap	Second-attempt trap, delayed trap	4–5
45–47	Combinations II	Read multiple lies in one room	Camera trick + morph + fake door	5
48–50	Gauntlet	Everything you've learned, fast	Mixed catalogue, tight spacing	5

Why this curve works. Each band introduces exactly one idea, gives the player a few levels to internalize it, then folds it into combinations. The "second-attempt trap" band (41–44) is the signature twist: by then the player trusts that a known level is safe, so a trap that only triggers on the *second* run lands as the funniest, fairest-in-hindsight betrayal in the game. Levels 51+ (Beta/ post-launch, Section 5) keep recombining the catalogue and add seasonal trap variants.

Part VI · Audio, Interface & Art Direction

Visual presentation in *PLUMMET* is a design tool, not decoration. The plain art makes traps read as fair in hindsight; the snappy audio and haptics make failure feel funny instead of punishing; the UI exists to get the player back into the action in under a second.

17. Audio and Haptics

Audio carries most of the game's comedy and feedback. The set is small, punchy, and pooled so overlapping deaths never crackle or stall.

17.1 Sound set

- **Jump** — light, short.
- **Land** — soft thud, pitched slightly by fall speed.
- **Death** — the comedic centerpiece: a short, exaggerated stinger (squish/boing/poof) that makes dying enjoyable.
- **Trap trigger** — a distinct cue per trap *family* so the ear learns "that sound = that lie."
- **Door** — a satisfying open/clear cue.
- **Level complete** — a brief triumphant flourish.
- **UI click** — crisp, immediate tap feedback.
- **Comedy stingers** — occasional variation on death/fake-victory to keep repetition fresh.

17.2 Haptics

Haptics reinforce physical feedback on mobile:

- **Death** — a sharp, short buzz that punctuates the failure (the tactile half of the comedy beat).
- **Trap activation** — a light tick when a major trap fires, so the player *feels* the betrayal.
- Keep haptics **short and rare**; constant vibration drains battery (Section 21) and annoys. All haptics route through a `HapticsService` behind an interface, gated by the user's haptics toggle (Section 18) and disabled automatically on unsupported devices.

17.3 Audio architecture and mobile constraints

- **Audio pooling.** Play SFX through a pooled set of `AudioSource`s (Section 21) so rapid deaths/retries never allocate or exceed voice limits.
- **Compression.** Compress SFX (Vorbis/ADPCM as appropriate) and set short clips to **Decompress on Load** and music to **Streaming**; keep import settings mobile-friendly (Section 21).
- **Avoid overlap pileups.** Cap simultaneous voices and coalesce identical SFX fired in the same frame (one death sound, not five).
- **Sound priority.** Death and level-complete cues take priority over ambient/UI sounds when the voice budget is tight.
- **User settings.** A sound toggle and a haptics toggle (persisted in `SaveData`, Section 20) are mandatory; both are honored everywhere via the audio/haptics services, never by muting individual sources ad hoc.

18. UI/UX Design

The UI philosophy is **minimal chrome, maximum return-to-play speed**. Menus are clean and few; in-level UI is barely-there; the path from death to the next attempt has essentially zero friction.

18.1 Screens and elements

- **Main menu** — title, big **Play** (continue at current level), **Level Select**, **Settings**, and a small **Privacy** link (Section 27) and **Credits**.
- **Level selection** — a grid of unlocked levels with completion ticks; locked levels are visibly gated (Section 13).
- **Pause menu** — **Resume**, **Restart**, **Quit to menu**, sound/haptics toggles.
- **Settings** — sound toggle, haptics toggle, language (if localized), privacy/consent controls (Section 27), credits, and a reset-progress option (guarded by confirmation).
- **In-level HUD** — as little as possible: the mobile controls, a small pause button, and an unobtrusive level label.
- **Mobile controls** — left/right movement buttons and a jump button (Section 14), placed for two-handed landscape play.
- **Loading screen** — a quick, clean wipe (Section 12).

18.2 Touch targets and placement

- **Touch target size.** Interactive controls are at least ~9–10 mm (~48 dp) on the smallest supported screen; movement and jump buttons are larger and sit in the natural thumb arcs (move on the lower-left, jump on the lower-right).
- **Safe area.** All controls and the pause button respect the device safe area (Section 6) so notches and gesture bars never overlap inputs.
- **Thumb occlusion.** Keep critical readouts away from where thumbs rest; the bottom corners are for controls, the top is for the (minimal) HUD.

18.3 Fast-restart UX and friction reduction

This is the most important UX rule in the game. After a death:

- Respawn is **immediate** (Section 14); the death stinger plays *over* the start of the next attempt, not as a blocking gate.
- There is **no mandatory tap, no "you died" modal, no ad** between death and retry (ad rules in Section 25).
- The restart button (in pause and as a quick gesture) re-arms the level instantly via the reset path (Section 12).
- The emotional goal: a death should feel like a comma, not a full stop.

18.4 Accessibility

- Honor OS **reduce-motion** by toning down camera shake/zoom tricks where they would harm comfort (without removing the gameplay-relevant information).
- Maintain high **contrast** between *Blip*, hazards, and background (Section 19) so traps are legible.
- Provide **haptics and sound toggles** for players who need them off.
- Keep text large and legible (TextMeshPro, Section 6); avoid conveying critical state by color alone.

19. Art Direction

The art is intentionally plain. This is a feature: a clean, low-detail world means there is nowhere for a trap to "cheat" — once revealed, every hazard is obviously readable, which is what makes the game feel fair. Visual noise would make tricks feel like unfairness rather than deception.

19.1 Style summary

- **Character:** a plain black silhouette (*Blip*) — no face, no eyes, no mouth, blocky human-ish shape.
- **Platforms:** simple rectangles, flat fills, minimal edge detail.
- **Spikes:** simple triangles, instantly recognizable.
- **Doors:** simple rectangular frames, one clear "open" state.
- **Backgrounds:** muted gradients, low contrast among themselves, high contrast against gameplay elements.
- **Overall:** strong figure/ground contrast, minimal visual noise, generous negative space.

19.2 Technical art specs

- **Character sprite size:** small — on the order of 16×16 to 24×24 px source art at 16 PPU; *Blip* reads as a compact block on screen.
- **Animation frame counts:** 1 idle, 2–4 walk, 1 jump, 1 hurt/death (Section 14) — deliberately tiny.
- **Pixels per unit:** 16 project-wide (Section 6), Point filter, no mips.
- **Platform style:** flat rectangles in one or two muted tones; optional 1-px top highlight for readability.
- **Spike style:** solid triangles with a single accent; the silhouette alone communicates danger.
- **Door style:** a framed rectangle that contrasts with platforms; clearly "the goal."
- **Background style:** two- or three-stop vertical gradients, desaturated, never competing with foreground.
- **UI button style:** flat, rounded rectangles, large hit areas, one accent color (Section 18).

19.3 Color palette and readability

Use a small, disciplined palette: a near-black for *Blip* and hazards' cores, a bright accent (for example a warm orange) reserved for danger/important cues, muted mid-tones for platforms, and desaturated gradients for backgrounds. The readability rules: **(1)** the player and lethal hazards always have the highest contrast on screen; **(2)** the danger accent color is used *only* for danger, so players learn it instantly; **(3)** decorative background elements never use the danger accent; **(4)** every trap, once sprung, is visually distinct from safe geometry on the retry.

19.4 Character direction (reference)

Blip — character art specification

```
Player:
- Black silhouette
- Square head
- Small rectangular body
- Simple block arms
- Simple block legs
- No eyes
- No mouth
- No accessories unless needed
- 2–4 frame walk animation
- 1 frame idle
- 1 frame jump
- 1 frame hurt/death
```

Blip's facelessness is also a branding and merch asset: a simple, recognizable silhouette is cheap to animate, instantly readable at any size (including the store icon, Section 26), and trivial to reskin for cosmetic monetization (Section 25).

Part VII · Data, Performance & Quality

This part covers the systems that keep the game persistent, fast, observable, and correct: how progress is saved, how the game hits its frame and memory budgets on low-end phones, how

analytics turns deaths into balance decisions, and how testing and debug tooling keep a fast-growing level catalogue from regressing.

20. Save System

The save system is small but must be bulletproof: players will have dozens of hours and hundreds of levels of progress, and a corrupted save is a one-star review. The interface lives in `Game.Core.Contracts`; the JSON-and-file implementation lives in `Game.Core` (Section 9).

20.1 What is saved

- **Current/highest unlocked level** — the single most important value (drives "Continue").
- **Completed levels** — for the level-select ticks and completion stats.
- **Death count** (total and optionally per level) — for analytics and player-facing stats.
- **Best times** (optional, per level) — for leaderboards/challenge (Section 5).
- **Settings** — sound and haptics toggles, language.
- **Save version** — schema version for migration.

20.2 Save data model (Core.Contracts)

The model is a plain serializable C# type with no engine dependency, so it can be unit-tested and migrated in pure code.

Assets/_Project/Code/Core/Contracts/Save/SaveData.cs · asmdef: Game.Core.Contracts

```
using System;

namespace Game.Core.Contracts.Save
{
    [Serializable]
    public sealed class SaveData
    {
        public int Version;
        public int HighestUnlockedLevel;
        public int TotalDeaths;
        public bool SoundEnabled;
        public bool HapticsEnabled;
    }
}
```

20.3 Save service interface (Core.Contracts)

The shared contract the whole game depends on. Callers never know whether the data is JSON on disk, in `PlayerPrefs`, or in the cloud.

Assets/_Project/Code/Core/Contracts/Save/ISaveService.cs · asmdef: Game.Core.Contracts

```
namespace Game.Core.Contracts.Save
{
    public interface ISaveService
    {
    }
```

```

    SaveData Load();
    void Save(SaveData data);
    void DeleteSave();
}
}

```

Why the split. `ISaveService` and `SaveData` belong in `Game.Core.Contracts` because the *rest of the game* depends on the promise of saving, and that promise must be engine- and storage-agnostic and unit-testable. `JsonSaveService` belongs in `Game.Core` because serialization format and file IO are implementation details that may change (JSON today, binary or cloud tomorrow) without any caller changing. This is the shared-contract backbone of Section 9 applied to persistence.

20.4 Versioning, migration, and corruption handling

- **Versioning.** Every write stamps `Version`. On load, if the stored version is older, `SaveMigrationService` upgrades the data forward, field by field, to the current schema (Section 7) — never assume the on-disk shape matches the current code.
- **Migration.** Keep migrations small and ordered ($v1 \rightarrow v2 \rightarrow v3$); each migration is pure and unit-tested (Section 23).
- **Corruption handling.** Wrap load in try/validate: if the file is unreadable or fails validation, fall back to a fresh default `SaveData`, optionally preserving a backup of the corrupt file for diagnostics, and never crash. Write atomically (write to a temp file, then move/replace) so a kill mid-write cannot truncate the live save.
- **Path.** Local saves live under `Application.persistentDataPath` (the one place the save service touches `UnityEngine`).
- **Cloud (optional).** A future cloud implementation provides the same `ISaveService` (or sits behind it) so cloud sync is additive, not a rewrite.
- **Validation.** After load/migrate, clamp/validate values (`HighestUnlockedLevel` within known range, toggles boolean) before handing data to the game.

21. Mobile Optimization Techniques

The performance target is **60 FPS on mid-range phones with a 30 FPS floor on low-end**, fast restarts, small builds, and low battery draw. The art style helps enormously (few, flat sprites), but the engineering discipline below is what keeps it there as content grows.

21.1 Rendering and textures

- **Sprite atlasing.** Group sprites into per-category Sprite Atlases (characters, platforms, traps, UI) so the 2D renderer batches draws (Section 6). This is the biggest single draw-call win.
- **Texture compression.** Use **ASTC** on both platforms for non-pixel UI/large textures; keep tiny pixel sprites uncompressed (Section 6) but atlased.
- **Pixel-art settings.** Point filter, no mips, Full Rect (Section 6).
- **Avoid large textures.** No 4K atlases; keep atlas pages reasonable and split by category.

- **Reduce overdraw.** Flat opaque sprites, minimal transparent layering, no full-screen translucent stacks; check with the overdraw view.

21.2 Memory and allocation

- **Object pooling.** Pool anything spawned frequently — death VFX, dust, particle bursts (Section 11). Never `Instantiate / Destroy` per death.
- **Avoid runtime allocations.** No per-frame `new`, no LINQ in hot paths, cache arrays/lists, avoid boxing (watch the analytics params dictionary, Section 22). Aim for **zero GC allocations per frame** during steady gameplay.
- **Avoid unnecessary `Update`.** Prefer the few `VContainer` tick entry points (Section 10) over dozens of `MonoBehaviour.Update` methods; empty/!near-empty `Update`s still cost.

21.3 Physics and simulation

- **Physics2D layer filtering.** Trim the collision matrix so only meaningful layer pairs test (Section 6).
- **Collision matrix optimization.** Disable Player ↔ Player, VFX ↔ everything, etc.
- **Rigidbody2D body types.** Static for non-moving geometry, Kinematic for scripted movers (most traps at rest), Dynamic only when real physics is needed (a falling platform *after* it drops, Section 15). Static/Kinematic bodies are far cheaper than Dynamic.

21.4 UI, audio, and content streaming

- **UI Canvas optimization.** Split static and dynamic UI onto separate canvases so a changing element does not rebuild the whole canvas; disable raycasts on non-interactive graphics.
- **Audio compression.** Compress and set appropriate load types (Section 17); cap voices.
- **Addressables for level packs.** Stream level packs via Addressables (Section 6) so the base build stays small and post-launch packs download on demand (Section 25).
- **Scene loading.** Keep the four scenes light (Section 12); load levels as prefabs/Addressables, not as scenes, for instant restart.
- **Build size reduction.** Managed code stripping (IL2CPP), strip unused packages, compress textures/audio, keep third-party SDKs minimal (Section 6).

21.5 Profiling and verification

- **Memory profiling.** Use the Memory Profiler to catch leaks across many restarts (a leak shows as steadily climbing memory per retry — exactly what scoped level lifetimes prevent, Section 12).
- **Frame Debugger.** Verify sprites are batching and draw calls stay low.
- **Unity Profiler.** Watch CPU (scripts, physics), GC allocations, and rendering on-device, not just in the editor.
- **Battery.** Cap frame rate explicitly (`Application.targetFrameRate`), avoid busy-waiting, keep haptics rare (Section 17) — battery drain is a real review driver on this genre.
- **Low-end device testing.** Maintain at least one genuinely low-end Android device in the test matrix (Section 23) and validate the 30 FPS floor there.

21.6 Performance budget

Metric	Target	Low-end ceiling	Notes
Frame rate	60 FPS	30 FPS floor	Explicit <code>targetFrameRate</code> ; VSync off
Restart time	< 0.2 s	< 0.3 s	Reset path, not scene reload (Section 12)
Level load time	< 0.6 s	< 1.0 s	Prefab/Addressable load
Memory (runtime)	< 200 MB	< 250 MB	Verified across many restarts
Build size	< 80 MB	< 100 MB	Base build; packs streamed
Draw calls	< 40	< 80	Achieved via atlasing
GC alloc / frame	0 B	near 0 B	Steady-state gameplay

21.7 Object pool interface (Core.Contracts)

A minimal generic pool contract. Concrete pools live in `Game.Core` (Pooling).

Assets/_Project/Code/Core/Contracts/Pooling/IObjectPool.cs · asmdef: `Game.Core.Contracts`

```
namespace Game.Core.Contracts.Pooling
{
    public interface IObjectPool<T>
    {
        T Get();
        void Release(T item);
    }
}
```

21.8 Pooled effect service outline

The death/VFX service gets effects from a pool, positions and plays them, and returns them when finished — no per-death allocation.

Assets/_Project/Code/Core/Runtime/Pooling/EffectPoolService.cs · asmdef: `Game.Core`

```
using UnityEngine;

namespace Game.Core.Pooling
{
    public sealed class EffectPoolService
    {
        public void PlayDeathEffect(Vector3 position)
        {
            // Get effect from pool.
            // Move effect to position.
            // Play animation or particle effect.
            // Return effect to pool when finished.
        }
    }
}
```

22. Analytics and Telemetry

Analytics exists for one practical purpose in this game: **find unfair levels and drop-off points and fix them**. A troll platformer lives or dies on the surprise-vs-fairness balance (Section 2), and the only way to verify that balance at scale is to measure how players actually fail.

22.1 Event set

- `game_launched`
- `level_started` (level id)
- `level_completed` (level id, time, deaths-this-attempt)
- `level_failed` (level id)
- `player_died` (level id, position/zone)
- `death_reason` (level id, trap id / cause)
- `trap_triggered` (trap id, level id)
- `retry_count` (level id)
- `time_to_complete` (level id)
- `level_abandoned` (level id, where)
- `ad_watched` (placement, rewarded/interstitial)
- `hint_used` (level id)
- `settings_changed` (setting, value)

22.2 How the data drives design

- **Balancing.** A level whose death-to-completion ratio is far above its neighbors is too hard *or* unfair; the `death_reason` / `trap_triggered` breakdown says which trap is responsible (Section 16).
- **Detecting unfair levels.** Many deaths with *no* eventual completion (high abandon) flags a level that reads as unfair rather than tricky — a fairness-rule violation (Section 2).
- **Drop-off points.** `level_abandoned` aggregated by level reveals exactly where players quit the campaign, prioritizing fixes.
- **Privacy.** Analytics is **event-and-aggregate only** — no personal data, no precise location, minimal identifiers — and is gated by consent where required (Section 27). Players can opt out in Settings; the service honors it.

22.3 Analytics service interface (Core.Contracts)

A vendor-agnostic contract so the chosen SDK (added in Alpha/Beta, Section 5) is one swappable implementation in `Game.Core`.

Assets/_Project/Code/Core/Contracts/Analytics/IAalyticsService.cs · asmdef: `Game.Core.Contracts`

```
using System.Collections.Generic;

namespace Game.Core.Contracts.Analytics
{
    public interface IAnalyticsService
    {
        void TrackEvent(string eventName);
    }
}
```



```

        void TrackEvent(string eventName, IReadOnlyDictionary<string, object> parameters);
    }
}

```

Gameplay code raises typed events (Section 15); a thin subscriber translates them into `TrackEvent` calls. Keeping that translation in one place (rather than scattering `TrackEvent("player_died")` through gameplay) keeps event names consistent and makes the parameter dictionaries easy to pool/avoid-boxing (Section 21).

23. Testing Strategy

The testing strategy is tuned to the architecture: because `Game.Core.Contracts` and the logic-only features are engine-free (Section 8), the bulk of logic is covered by **fast EditMode tests** that run headless in CI in seconds, with a thinner layer of PlayMode tests for integration.

23.1 Test types

- **Unit / EditMode tests** — pure C# logic: level progression/unlock rules, save migration, trap state transitions, movement math (coyote/buffer windows with a fake clock). Fast, no scenes.
- **PlayMode tests** — integration in a real scene: player actually moves and dies, a trap actually triggers and resets, a level actually completes. Slower, fewer.
- **Input tests** — drive `IPlayerInput` with a `FakePlayerInput` (Section 14) to assert movement responses deterministically.
- **Trap reset tests** — assert that after `Activate()` then `ResetTrap()`, a trap is byte-for-byte back to its armed state (the determinism guarantee behind restart, Section 12/15).
- **Save/load tests** — round-trip `SaveData`, plus migration from each old version to current.
- **Level completion tests** — completing a level unlocks the next and persists.
- **Death-reason tests** — the correct cause is reported for a given trap hit (feeds analytics correctness, Section 22).
- **Performance tests** — profiler-marker/allocation assertions on hot paths; verify zero steady-state GC (Section 21).
- **Device tests** — manual + automated runs across the device matrix (low-end Android, mid Android, range of iPhones).
- **Regression tests** — a fixed suite re-run before every release (Section 26) so a new trap never silently breaks an old level.

23.2 Example EditMode test

A representative pure-C# progression test — no scene, no `MonoBehaviour`, milliseconds to run.

```

Assets/_Project/Code/Features/Progression/Tests/LevelProgressTests.cs · asmdef:
Game.Features.Progression.Tests

using NUnit.Framework;
using Game.Features.Progression;

namespace Game.Features.Progression.Tests

```

```

{
    public sealed class LevelProgressTests
    {
        [Test]
        public void Completing_Level_Unlocks_Next_Level()
        {
            var progress = new LevelProgress(highestUnlockedLevel: 1);

            progress.CompleteLevel(1);

            Assert.AreEqual(2, progress.HighestUnlockedLevel);
        }
    }
}

```

Why pure-C# unit tests matter. This test constructs a progression object with `new`, exercises it, and asserts — no Unity scene, no play mode, no container. That is only possible because `LevelProgress` lives in the engine-free `Game.Features.Progression` assembly (Section 8). The payoff: hundreds of these run in CI on every push (Section 26) in seconds, catching progression/save/trap-logic regressions long before a build is made. Reserve the slow `PlayMode` tests for things that genuinely need a scene.

24. Debug and Developer Tools

A troll platformer is authored and QA'd by repeatedly testing tricks, so good developer tooling pays for itself immediately. The rule: **rich tools in development builds and the editor; nothing exploitable or performance-costing in shipping player builds.**

24.1 Tooling

- **Level skip** — jump to any level (editor / dev build only).
- **Restart level** — instant re-arm hotkey.
- **Unlock all levels** — for testing late content (dev only).
- **Reset save** — wipe to fresh `SaveData` (dev only; player-facing version is the guarded Settings option, Section 18).
- **God mode** — ignore death, to walk a level and inspect traps (dev only).
- **Trap visualization** — draw trigger zones, armed/triggered state, and reset bounds via gizmos.
- **Trigger visualization** — show all `TrapTrigger` colliders and which trap each drives.
- **Death-reason logger** — log every death's cause to validate analytics mapping (Section 22).
- **FPS overlay** — on-screen frame rate (dev build).
- **Memory overlay** — live memory readout to catch leaks across restarts (Section 21).
- **Level validation tool** — the editor/CI validator (Section 16): checks spawn, reachable door, trap wiring/reset, jump-arc feasibility.
- **Screenshot capture** — one-key clean screenshots for store assets (Section 26).
- **Build information overlay** — version, build number, commit hash, environment (dev build).

24.2 Editor-only vs. development build

- **Editor-only** (custom windows, gizmos, validators, cheat menus): compiled in `Game.Editor` (Section 8) and never in any player build.
- **Development-build only** (FPS/memory/build-info overlays, death logger): guarded by `Debug.isDebugEnabled` / `#if DEVELOPMENT_BUILD` so they vanish from release builds.
- **Never in release**: god mode, unlock-all, level-skip, and anything that bypasses progression or costs frame time. These are stripped, not merely hidden, to avoid exploits and to protect the performance budget (Section 21).

Part VIII · Monetization, Release & Compliance

This part turns a finished game into a shipped, sustainable product: how it earns money without betraying the fast-retry promise, how it gets built and signed and pushed to both stores, and how it stays on the right side of privacy law and store policy.

25. Ads and Monetization

Monetization must never fight the core loop. The entire appeal of *PLUMMET* is that dying is fast and funny (Section 2); an ad after every death would destroy that in a single session. The model is therefore **light, optional, and player-respecting**.

25.1 Revenue options

- **Rewarded ads for hints** — a player stuck on a level can *opt in* to watch an ad for a hint (a ghost showing the safe path, or revealing one trap). Always optional, never forced.
- **Interstitials, sparingly** — a brief interstitial after a *milestone* (e.g. every several completed levels), never tied to death and never in the opening levels.
- **Remove-ads IAP** — a one-time purchase that removes interstitials (rewarded hints remain available by choice).
- **Cosmetic skins** — reskins of *Blip* (Section 19); pure cosmetics, no gameplay advantage.
- **Level packs** — premium or ad-supported additional level packs delivered via Addressables (Section 21).
- **Challenge mode** — optional timed/streak modes for engagement (and a natural rewarded-ad surface).

25.2 Ethical monetization rules (hard constraints)

- ☐ **No ads after every death.** Death → retry stays instant and ad-free, always (Section 18).
- ☐ **Do not interrupt the first levels.** No interstitials during onboarding; let players fall in love with the loop first.
- ☐ **Rewarded ads are strictly optional.** The game is fully completable without watching a single ad.
- ☐ **The retry loop stays fast.** No monetization step may add friction to restart.

- ☐ **Ads never make the game feel unfair.** No "pay/watch to continue" gates, no ad-driven difficulty.
- ☐ **Frequency capping.** Interstitials are capped per session and per time window, tuned via A/B testing (Section 5) against retention.

The commercial bet (Section 1) is that high retention plus a long content tail beats aggressive short-term ad revenue. Ads are implemented behind an `IAdsService` interface (Section 8) so the vendor SDK is a swappable adapter, added in Beta (Section 5) and kept out of early builds.

26. Build and Deployment Pipeline

The pipeline produces signed, store-ready Android and iOS builds reproducibly, ideally from CI, with versioning and crash reporting wired in. The goal is that "make a release build" is one command/one pipeline run, not a fragile manual ritual.

26.1 Platform builds

- **Android.** IL2CPP / ARM64 (Section 6), App Bundle (`.aab`) for Play, R8/managed stripping on. Validate on the low-end device (Section 21).
- **iOS.** Unity exports an Xcode project; build/archive via Xcode (or `xcodebuild` in CI) with the correct team, provisioning profile, and signing identity.

26.2 Signing and identifiers

- **Keystore (Android).** Generate once, store the keystore and credentials in a secure secret store (never in the repo), and back it up — losing it means you can never update the app under the same listing.
- **Bundle identifier.** `com.[studio].plummet` on both platforms, fixed before first submission.
- **App signing.** Use Play App Signing (Google manages the app key, you keep an upload key) and iOS automatic or manual signing with managed certificates.

26.3 Versioning and build numbers

- **Version name** follows semver (`1.0.0`) and is shown to players.
- **Build/version code** is a monotonically increasing integer, ideally stamped automatically by CI from the build number/commit count (a `BuildTools` script, Section 7). Each store upload must have a higher code than the last.
- **Changelog.** Maintain a human changelog per release (Section 33) and reuse it for store "what's new" text.

26.4 Testing tracks and release flow

- **iOS:** push to **TestFlight** for internal/external testing before App Store submission.
- **Android:** use **Internal testing** → **Closed** → **Open/Production** tracks on Play; promote a build through tracks rather than uploading fresh ones.
- **Release branches.** Cut `release/x.y.z` from `develop`; only stabilization fixes land on it; tag the exact commit that ships (Section 31).

- **Hotfix branches.** `hotfix/x.y.z` from `main` /tag for urgent post-launch fixes, merged back to both `main` and `develop`.

26.5 CI/CD

Automate with GitHub Actions (or equivalent): on push, run EditMode tests and the level validator (Sections 16, 23); on a release tag, build Android `.aab` and the iOS Xcode archive, stamp version/build numbers, and upload to the respective testing tracks. Keep signing secrets in the CI secret store. CI catching a broken level or failing test *before* a build is the cheapest possible bug fix.

Release pipeline (conceptual)

```
push → [CI] EditMode tests + level validation → green?
                                         | yes
tag vX.Y.Z → [CI] build .aab + iOS archive → stamp version/build
                                         → upload to Play Internal + TestFlight → QA sign-off
                                         → promote to Production / submit for review
```

26.6 Crash reporting and store assets

- **Crash reporting.** Integrate a crash/exception reporter (behind an interface, Section 8) to catch device-specific crashes the test matrix missed; disclose it in the privacy policy (Section 27).
- **Store metadata.** Title, subtitle, descriptions, keywords, category, contact and privacy URLs.
- **Screenshots.** Captured with the screenshot tool (Section 24) at required device sizes; show the *deception* hook (a clean room with a hidden danger) rather than spoiling tricks.
- **App icon.** The *Blip* silhouette (Section 19), legible at the smallest size.
- **Privacy policy & age rating.** Required by both stores (Section 27).

26.7 Release checklist

- ☐ Version name and build number bumped and stamped.
- ☐ EditMode + PlayMode tests and level validation green in CI.
- ☐ Signed `.aab` and signed iOS archive produced by CI.
- ☐ Crash reporting verified live in the build.
- ☐ Store metadata, screenshots, and icon uploaded for both stores.
- ☐ Privacy policy published and linked; age rating completed.
- ☐ TestFlight and Play internal testing builds validated by QA.
- ☐ Changelog / "what's new" written.
- ☐ Rollback/hotfix plan and on-call owner confirmed (Section 30).

27. Privacy, Compliance, and Store Requirements

Both Apple and Google gate releases on privacy disclosure and policy compliance, and analytics/ads bring real legal obligations. The guiding principle is **data minimization**: collect as little as possible, disclose all of it, and make tracking optional.

27.1 Requirements

- **Privacy policy.** A published, linked privacy policy is mandatory on both stores; it must accurately describe analytics, ads, and crash reporting data use.
- **Analytics disclosure.** Disclose what analytics events are collected (Section 22) and that they are aggregate/non-personal; offer an opt-out in Settings.
- **Ads disclosure.** Disclose the ad SDK and any identifiers it uses (Section 25); complete the stores' data-safety / privacy "nutrition label" forms accordingly.
- **Children's privacy.** If the game targets or appeals to children, comply with COPPA / Google's Families policy / age-appropriate design: no behavioral ad targeting to children, restricted data collection, and an appropriate content rating. Decide the target-age posture explicitly before launch — it changes ad configuration and store forms.
- **iOS App Tracking Transparency (ATT).** If any SDK uses the advertising identifier (IDFA) for tracking, present the ATT prompt and respect the user's choice; if you do not track, declare so and skip the prompt.
- **GDPR / CCPA.** For EU/California users, support consent (where ads/analytics require it), honor opt-out/erasure, and gate non-essential data collection behind consent. A lightweight consent flow on first launch (or via a CMP for ads) covers this.
- **Data minimization.** Do not collect names, contacts, precise location, or device identifiers you do not need. The save (Section 20) is local by default; analytics is event-only (Section 22).
- **Store age rating.** Complete IARC/age-rating questionnaires honestly (cartoon/comedic peril, no real violence); the rating must match actual content.
- **Crash reporting disclosure.** Disclose crash/diagnostic data collection (Section 26) in the policy and store forms.
- **Consent handling.** Where consent is required, the analytics/ads services must **default to off** until consent is given, and a settings control must let users change their mind later.

Practical posture for an indie launch. Minimize data, disclose everything, default tracking off where consent is required, keep the save local, and gate ad/analytics initialization behind the consent and ATT outcomes. This is both the compliant path and the one least likely to cause a store rejection (Section 30).

Part IX · Production Management

The plan for *building* the thing: a realistic phase-by-phase timeline for a 1–4 person team, who does what (including the solo case), and the risks most likely to sink the project with concrete mitigations.

28. Production Timeline

This timeline targets a small indie team of one to four people. Durations are ranges, not promises; the gates between phases ("exit criteria") matter more than the calendar. Content (level count) scales fastest in Alpha/Beta because the trap framework (Section 15) makes new levels cheap.

28.1 Phases

Phase 1 · Pre-production — 1–2 weeks. Deliverables: game design brief, technical architecture plan (this document), art direction, production schedule, prototype scope. Exit: scope agreed and architecture/skeleton committed (Part III).

Phase 2 · Prototype — 2–3 weeks. Deliverables: player controller, one basic level, a few basic traps, the restart loop, mobile controls. Exit: the core loop is fun to fail at on a real phone.

Phase 3 · MVP — 4–6 weeks. Deliverables: 10–20 levels, the VContainer architecture in place (Section 10), save system (Section 20), UI foundation (Section 18), basic audio (Section 17), an internal test build. Exit: a player can install, progress, and quit/resume cleanly.

Phase 4 · Vertical Slice — 3–4 weeks. Deliverables: a polished sample level pack, final visual direction (Section 19), final player feel (Section 14), the trap system finalized (Section 15), a performance test on-device. Exit: this *looks and feels* like the shipping game.

Phase 5 · Alpha — 6–8 weeks. Deliverables: 40–60 levels, most systems complete, analytics (Section 22), ads placeholder (Section 25), QA tools (Section 24), device testing (Section 23). Exit: feature-complete, content-substantial, instrumented.

Phase 6 · Beta — 4–6 weeks. Deliverables: 70–100 levels, full UI, full audio pass, ads integration, store preparation (Section 26), performance optimization to budget (Section 21). Exit: content-complete and store-ready.

Phase 7 · Release Candidate — 2–3 weeks. Deliverables: bug fixes (blocker/critical), final QA, app signing, store assets, privacy policy (Section 27), TestFlight and Play internal testing. Exit: a build you would ship, pending only final validation.

Phase 8 · Launch — 1 week. Deliverables: production release on both stores, monitoring (analytics + crash), hotfix readiness (Section 26). Exit: live and stable.

Phase 9 · Post-launch — ongoing. Deliverables: new levels/packs, bug fixes, balance updates driven by analytics (Section 22), monetization tuning (Section 25), analytics review. Exit: none — this is live operations (Section 5).

28.2 Milestone table

The full per-phase breakdown across disciplines. (Rendered landscape for width.)

Phase	Duration	Gameplay	Technical	Art	Audio	QA	Deliverables	Exit criteria
Pre-production	1–2 wk	Loop & pillars defined	Architecture plan, project skeleton (Part III)	Art direction set	Audio direction set	Test plan drafted	Brief, arch plan, schedule, prototype scope	Scope + architecture agreed
Prototype	2–3 wk	Move, jump, restart, 1 level	Player service, input abstraction	Placeholder Blip + tiles	Temp jump/death SFX	Loop "fun-to-fail" check	Playable core on device	Core loop is fun
MVP	4–6 wk	10–20 levels, basic traps	VContainer scopes, save system	First real sprites	Basic SFX set	Save/progress test pass	Internal build, resume works	Install → progress → resume clean
Vertical Slice	3–4 wk	Final feel, trap system done	Trap framework, level loading	Final visual direction	Audio sample pass	On-device perf test	Polished sample pack	Looks/feels shippable
Alpha	6–8 wk	40–60 levels	Analytics, ads placeholder, QA tools	Most art final	Most SFX final	Device matrix testing	Feature-complete build	Feature-complete + instrumented
Beta	4–6 wk	70–100 levels	Ads integration, save versioning	All art final	Full audio pass	Perf to budget, regression	Store-ready build	Content-complete + optimized
Release Candidate	2–3 wk	Final tuning	Signing, store builds, crash reporting	Store assets, icon	Final mix	Full QA pass (Section 34)	Signed RC on test tracks	Ship-quality, validation only
Launch	1 wk	—	Production push, monitoring	—	—	Smoke test live build	Live on both stores	Live and stable
Post-launch	ongoing	New packs, balance	Hotfixes, A/B tests	Seasonal/skins	New stingers	Ongoing regression	Updates & live ops	(continuous)

29. Team Roles and Responsibilities

The roles below describe a full small team; in practice an indie team wears several hats. The solo configuration that follows maps every role onto one person with a realistic sequencing strategy.

29.1 Roles

- **Game designer** — owns levels, traps, difficulty curve, and the surprise-vs-fairness balance (Sections 2, 16); defines what is fun.
- **Unity developer** — implements gameplay systems (player, traps, level runtime) against the architecture (Parts IV–V).
- **Technical architect** — owns the layering, assembly boundaries, DI composition, and coding standards (Sections 8–11, 31); guards against architectural drift.
- **Pixel artist** — produces *Blip*, platforms, traps, backgrounds, VFX, and the icon to the art spec (Section 19).
- **UI/UX designer** — designs menus, mobile controls, and the friction-free death/retry flow (Section 18).
- **Sound designer** — creates the comedic SFX set, stingers, and haptics feel (Section 17).
- **QA tester** — runs the device matrix, regression suite, and the production checklist; verifies fairness empirically with analytics (Sections 23, 34).
- **Producer** — owns the schedule, scope, risk register, and store submission timeline (Sections 28, 30).
- **Marketing / store release manager** — owns store listings, screenshots, ASO, launch, and post-launch comms (Sections 26, 27).

29.2 Solo developer configuration

One person can ship this game; the trick is sequencing, not heroics. Map the roles onto phases (Section 28) so you only wear one or two hats at a time:

- **Pre-production / Prototype:** wear *architect* + *developer* + *designer*. Lock the architecture (Part III) once so future-you never refactors assemblies, and prove the loop is fun before investing in content or art.
- **MVP / Vertical Slice:** wear *developer* + *designer*, then a focused *art* + *audio* + *UX* pass to set the final look/feel on a small slice. Use placeholder art until the slice, then do one decisive art pass rather than redrawing repeatedly.
- **Alpha / Beta:** wear *designer* most days (this is where the cheap-to-author trap framework pays off — content is the bulk of the work) with periodic *developer* sprints for analytics/ads and an *optimization* pass.
- **RC / Launch:** wear *QA* + *producer* + *release manager*: run the checklist (Section 34), prepare store assets, and submit.
- **Throughout:** lean on the architecture and CI (Sections 23, 26) as your "second teammate" — automated tests and level validation catch the regressions a solo dev cannot manually re-check across hundreds of levels. Outsource selectively (a musician for the SFX set, a contractor for store art) where it unblocks the most time.

30. Risk Management

The risks below are the ones that most commonly sink games in this genre and team size. Each has an impact, a rough probability, a concrete mitigation, and an owner (mapped to Section 29 roles; in the solo case, all owners are you, but the *mitigation* still applies).

Risk	Impact	Probability	Mitigation	Owner
Scope creep	High	High	Fixed pillars (Section 2) + phase gates (Section 28); new ideas go to a post-launch backlog, not the current phase	Producer
Levels become unfair	High	Medium	Fairness rules (Section 16) + analytics death/abandon monitoring (Section 22); fix or cut outlier levels	Designer
Player frustration	High	Medium	Instant retry, funny death, "fair after discovery" (Sections 2, 18); playtest for rage-vs-fun	Designer / UX
Poor mobile performance	High	Medium	Performance budget + profiling on low-end device every phase (Section 21)	Unity dev
Too many ads	High	Medium	Hard ethical ad rules (Section 25); A/B test frequency against retention	Producer / Marketing
Trap system becomes hardcoded	High	Medium	"Levels are data, not code" rule (Section 15); review new level scripts in code review (Section 31)	Architect
Save bugs / corruption	High	Low–Med	Atomic writes, versioning + migration, corruption fallback, round-trip tests (Sections 20, 23)	Unity dev
Store rejection	Medium	Medium	Privacy policy, data-safety forms, ATT, honest age rating done early (Section 27); pre-submission checklist	Release manager
Device fragmentation	Medium	High	Maintained device matrix incl. genuine low-end Android; 30 FPS floor verified (Sections 21, 23)	QA
Lack of content	High	Medium	Trap framework makes levels cheap; front-load the catalogue, schedule content in Alpha/Beta (Sections 15, 28)	Designer
Art too plain / unreadable	Medium	Low–Med	Readability rules + contrast discipline (Section 19); playtest "was the trap fair in hindsight?"	Artist
Poor retention	High	Medium	Onboarding, difficulty curve, content tail, analytics-driven tuning (Sections 16, 22, 5)	Producer / Designer

The two biggest, watched continuously. *Scope creep* and *unfair levels* are the most likely killers. Scope is controlled structurally (pillars + phase gates); fairness is controlled empirically (every level's death-to-completion and abandon rates are dashboards, not opinions). If a level's numbers are bad, it gets fixed or cut — no level is sacred.

Part X · Engineering Standards & Process

How the team writes, reviews, versions, and documents the code so that quality is mechanical rather than heroic. These standards exist to keep the architecture in Parts III–IV intact as the project grows and people (or future-you) forget why a rule was made.

31. Coding Standards

Standards are not bureaucracy; they are the cheapest way to keep a codebase readable when it outgrows any one person's head. The rules below are deliberately strict where it protects the architecture (Sections 9–10) and relaxed everywhere else. When in doubt, optimise for the next person who reads the file.

31.1 Naming conventions

Follow standard .NET / Microsoft C# conventions. They are widely known, tooling enforces them, and new contributors need zero ramp-up.

Element	Convention	Example
Namespace	PascalCase, mirrors assembly + folder	Game.Features.Traps
Class / struct / record	PascalCase, noun	FallingPlatformTrap
Interface	PascalCase with I prefix	ISaveService
Method	PascalCase, verb phrase	ResetTrap()
Public property	PascalCase	HighestUnlockedLevel
Public field (data objects only)	PascalCase	public int Version;
Private field	_camelCase	_elapsed
Local variable / parameter	camelCase	deltaTime
Constant	PascalCase	MaxBufferFrames
Enum type / members	PascalCase	GameState.Playing
Async method	PascalCase + Async suffix	LoadLevelAsync()
Type parameter (generics)	T prefix	IObjectPool<T>
ScriptableObject asset	PascalCase + type suffix	Level_07.asset, PlayerMovementConfig.asset
Boolean	reads as a question	IsGrounded, HasJumpBuffered

Additional rules: no Hungarian notation (`strName`); no abbreviations except universally understood ones (`id`, `ui`, `db` are fine, `plyr` is not); never use single-letter names except loop counters (`i`, `j`) and lambda parameters; never prefix interfaces onto concrete names to disambiguate

(`PlayerMovementServiceImpl` is banned — the interface is `IPlayerMovementService`, the class is `PlayerMovementService`).

31.2 Namespace and file organisation

- **One public type per file**, and the file name equals the type name (`FallingPlatformTrap.cs`). Small private helpers or nested types may share the file.
- **Namespace mirrors the assembly and folder**. A file at `Assets/_Project/Code/Features/Traps/Runtime/FallingPlatformTrap.cs` in assembly `Game.Features.Traps` is in namespace `Game.Features.Traps`. The mapping is mechanical, so anyone can locate a type from its fully-qualified name (Sections 7–8).
- **Usings at the top, sorted**, `System.*` first, then third-party, then `Game.*`. Enable "remove unused usings" on save.
- **No global using of UnityEngine in Game.Core.Contracts or logic-only features**. Those assemblies do not reference the engine (Section 8), so the using would not even compile — by design.
- **File-scoped namespaces** (`namespace Game.Features.Traps;`) are preferred in new files to reduce nesting; block-scoped is acceptable in existing files. Pick one per file, never mix.

31.3 MonoBehaviour rules

Monobehaviours are the bridge between Unity and our code, not the place our logic lives (Section 9). Keep them thin.

- A MonoBehaviour **adapts**: it reads engine input/lifecycle and forwards to an injected service, or it renders state pushed to it. It does not contain gameplay rules, scoring, save logic, or branching game flow.
- **No FindObjectOfType, GameObject.Find, Camera.main in hot paths, or singleton lookups**. Dependencies arrive by injection (Section 10) or serialized references set in the Inspector. Hidden scene-graph lookups are a banned form of global state.
- **No public mutable fields for wiring** other than `[SerializeField] private`. Designers configure via the Inspector; code configures via injection.
- **Cache component references in Awake**, never re-`GetComponent` per frame.
- **Update / FixedUpdate stay tiny**. If a MonoBehaviour needs per-frame logic, prefer an injected service implementing `ITickable / IFixedTickable` (Section 10) so the logic is testable and the MonoBehaviour stays a shell. Empty Unity messages (`void Update() {}`) are deleted — Unity still calls them and they cost performance (Section 21).
- A MonoBehaviour that has grown past ~150 lines or has more than a couple of responsibilities is a refactor signal, not a normal state.

31.4 ScriptableObject rules

ScriptableObjects are our **configuration and content** mechanism (levels, trap configs, tuning — Sections 14–16).

- Use them for **immutable, designer-authored data**, never as runtime mutable singletons or a backdoor "global state" object. Treat a loaded SO as read-only at runtime; if you need to mutate, copy into a runtime struct/class.

- Keep them **data-only**: fields, validation in `OnValidate`, and pure read helpers. No gameplay behaviour, no scene references, no service calls.
- Provide `[CreateAssetMenu]` with a clear menu path so designers can author them without code.
- Validate aggressively in `OnValidate` (clamp ranges, warn on missing references) so bad data is caught at author time, not at runtime on a player's device.

31.5 Events, delegates, and messaging

- Prefer **C# event Action<T>** or an injected typed event/signal bus over `UnityEvent` for code-to-code communication; reserve `UnityEvent` for designer-wired Inspector hooks.
- Name events as past-tense facts (`LevelCompleted`, `PlayerDied`) and their raisers as `OnLevelCompleted` only for the handler method. The event itself is the noun-phrase fact.
- **Always unsubscribe**. Subscribe in `OnEnable` /construction, unsubscribe in `OnDisable` / `Dispose`. Unmatched subscriptions are the most common Unity memory leak and a frequent cause of "ghost" callbacks after a scene reload (Section 12).
- Keep event payloads small and immutable. Never pass mutable engine objects through events across feature boundaries.

31.6 Asynchronous code

- Use **async / await with UniTask** (or `Awaitable` in Unity 6) for asynchronous flows — scene loads, save I/O, ad calls. Avoid coroutines for anything that returns a value or needs composition; reserve coroutines for simple frame-based animation where they read clearly.
- **Suffix async methods with Async** and return `UniTask / UniTask<T>` (avoid `async void` except for top-level event handlers).
- **Honour cancellation**. Pass a `CancellationToken` through async chains and cancel on scene teardown so a level reload never resumes a half-finished load (Sections 12, 20).
- Never block on async with `.Result / .Wait()` — it deadlocks on Unity's single-threaded context.
- Keep file and network I/O (save, analytics, ads) off the critical path; the retry loop must never `await` disk (Sections 2, 20).

31.7 Error handling

- **Fail loud in development, degrade gracefully in production**. A corrupt save throws in the editor (so we see it) but falls back to a fresh `SaveData` on a player's device (Section 20).
- Use exceptions for genuinely exceptional conditions, not control flow. Validate inputs at boundaries (implementations, deserialisation) and keep the Core and feature logic operating on already-valid data.
- **Never swallow exceptions silently** (`catch {}`). Catch specifically, log with context (Section 31.8), and either recover meaningfully or rethrow.
- Guard the player experience: an analytics or ads failure must never crash gameplay or block a restart. Those adapters catch their own errors and report them, the game continues.

31.8 Logging

- Route logs through a small injected `ILogService` (or wrap `Debug.Log`) so verbosity is controllable and logging can be compiled out of release.

- **Strip or gate verbose logs in release builds.** Wrap dev-only logging in `[Conditional("DEVELOPMENT_BUILD")]` or `#if` so per-frame logs never ship (string allocation in `Update` is a real cost — Section 21).
- Log with **context, not vibes**: include the level id, state, or entity, not just `"error"`. A log you cannot act on is noise.
- Reserve `LogError` / `LogException` for real problems so CI and crash dashboards (Section 26) stay signal-rich. Never use `LogError` for expected conditions.

31.9 Comments and XML documentation

- **Code says *what*; comments say *why*.** Don't narrate the obvious (`// increment i`); explain non-obvious decisions, trade-offs, and the reason a tricky bit exists.
- **XML doc comments (`/// <summary>`) on all public interfaces and their members** in `Game.Core.Contracts` — these are the contracts the rest of the game depends on (Sections 8–9), and IDE tooltips make them pay for themselves.
- Public types in Core and feature assemblies get a one-line summary; private implementation detail rarely needs doc comments if the names are good.
- Use `// TODO: / // HACK:` with an owner or issue reference so they're greppable and don't rot. A `HACK` without a reason is a future bug.
- Delete commented-out code. Git is the history (Section 32).

31.10 Testing expectations

Detailed strategy is in Section 23; the standards that bind day-to-day coding are:

- **Core contracts and feature logic ship with EditMode tests.** New use cases, progression rules, save migration, and trap state machines are covered by fast, engine-free tests (Section 23).
- **A bug fix includes a failing test first**, then the fix. This prevents regressions and documents the bug.
- Keep code testable by construction: depend on interfaces, inject dependencies, keep logic out of `MonoBehaviours` (Sections 9–10). If something is hard to test, that is usually a design smell, not a testing problem.
- CI runs the EditMode suite on every pull request (Section 26); a red build blocks merge.

31.11 Architectural rules — the non-negotiables

These restate the spine of Parts III–IV as bright lines that code review (Section 31.12) enforces. They are the rules most likely to be violated under deadline pressure, and the ones most expensive to fix later.

- ☐ **No static managers or singletons for game systems.** No `GameManager.Instance`, no static service locators. Systems are registered in a `LifetimeScope` and injected (Section 10). Statelessness and testability depend on this.
- ☐ **No God classes.** A type has one reason to change. A class accreting unrelated responsibilities (input + scoring + save + UI) is split before merge (Section 9).
- ☐ **No fat MonoBehaviours.** `MonoBehaviours` adapt and render; logic lives in injected services (Section 31.3).

- ☐ **No business logic in UI.** Presenters translate state to views and forward user intent; rules, scoring, and progression live in the relevant feature's logic, not the UI feature (Sections 9, 18).
- ☐ **No hidden scene dependencies.** No `Find / FindObjectOfType / Camera.main` -in-hot-path/singleton lookups for wiring. Dependencies are explicit (inject or serialize).
- ☐ **Features depend on Core, never on each other.** A feature assembly must not reference a sibling feature; cross-feature collaboration goes through Core contracts and the signal bus (Sections 8–9). Core never depends on a feature.
- ☐ **Core.Contracts stays engine-light.** No `UnityEngine` in the shared contract surface or logic-only features (Section 8), so the testable core stays provably portable.
- ☐ **Constructor injection by default.** Method/property injection only where the engine forces it (MonoBehaviours). Dependencies are visible in the signature, not hidden in the body (Section 10).
- ☐ **Composition happens only in `Game.App`.** The `LifetimeScope`s in `Game.App` are the only place that wires Core services and installs features; each feature owns its own installer. Nothing else `new`s a service (Sections 8, 10).
- ☐ **Levels are data, not code.** New levels are authored as assets, not C# (Sections 15–16). A level that needs a bespoke script is a missing trinket in the trap framework.

31.12 Code review checklist

Every pull request is read by one other person (or, solo, by you the next morning with fresh eyes — Section 29) against this list. Architecture first, because style is auto-formatted but design is not.

- ☐ Does it respect the feature and assembly rules (Sections 8–9)? No feature-to-feature references, no logic in the wrong slice.
- ☐ Any new static state, singleton, or hidden scene lookup? (Reject — Section 31.11.)
- ☐ Are MonoBehaviours thin and is logic in injected, testable services?
- ☐ Are dependencies injected via the composition root, not `new`'d or located ad hoc?
- ☐ Does new Core or feature logic come with EditMode tests, and do all tests pass in CI?
- ☐ Naming, namespaces, and file layout follow Sections 31.1–31.2?
- ☐ Are events unsubscribed and async operations cancellable (no leaks on reload)?
- ☐ Any per-frame allocations, `LogError` misuse, or work that belongs in a pool (Section 21)?
- ☐ Is configuration data (levels, tuning) authored as SO/data, not hardcoded?
- ☐ Is the change small enough to review in one sitting? (Prefer many small PRs — Section 32.5.)

32. Git Workflow

A lightweight, predictable Git process keeps history readable and releases reproducible. The model below scales from one developer to a small team without ceremony; the solo developer still benefits from branches and PRs because they make `git bisect`, rollbacks, and "what shipped in 1.3?" trivial.

32.1 Repository structure

- **One repository** containing the Unity project (`Assets/` , `Packages/` , `ProjectSettings/`) and the `Documentation/` folder (Section 33). A monorepo keeps code, content, and docs versioned together.
- Commit `Packages/manifest.json` and `packages-lock.json` so package versions (including VContainer) are pinned and reproducible across machines and CI (Section 26).
- Commit `ProjectSettings/` so editor and player settings are versioned. Never let two developers diverge on project settings silently.
- A correct `.gitignore` (Section 32.7) keeps generated folders out of the repo.

32.2 Git LFS for binary assets

Unity projects are binary-heavy; without LFS the repo bloats and diffs are useless.

- Track art, audio, and other large binaries with **Git LFS**: sprites/textures, audio clips, fonts, and any large media.
- Commit a `.gitattributes` so every clone uses LFS for those types from day one.

```
# Images
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.psd filter=lfs diff=lfs merge=lfs -text
*.aseprite filter=lfs diff=lfs merge=lfs -text
# Audio
*.wav filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
# Fonts & misc large binaries
*.ttf filter=lfs diff=lfs merge=lfs -text
*.otf filter=lfs diff=lfs merge=lfs -text
# Force text (never LFS) for code/meta so diffs stay readable
*.cs text
*.meta text
*.asmdef text
*.json text
```

Meta files are sacred. Always commit Unity `.meta` files alongside their assets, and never `.gitignore` them. A missing or conflicting `.meta` breaks references (broken prefabs, lost serialized links). Enable **Visible Meta Files** and **Force Text** serialisation in Project Settings so meta and scene/prefab files are diffable and merge-friendly (Section 6).

32.3 Branching model

A trunk-based model with short-lived branches. Keep `main` releasable and `develop` integrable at all times.

Branch	Purpose	Lifetime	Merges into
<code>main</code>	Always-shippable, tagged releases only	Permanent	—
<code>develop</code>	Integration branch; latest passing work	Permanent	<code>main</code> (at release)

Branch	Purpose	Lifetime	Merges into
<code>feature/<short-desc></code>	One feature or task (e.g. <code>feature/falling-platform-trap</code>)	Short (days)	<code>develop</code>
<code>bugfix/<short-desc></code>	Non-urgent fix during development	Short	<code>develop</code>
<code>release/<version></code>	Stabilise a version: bug fixes, store prep, version bump	Short	<code>main</code> and <code>develop</code>
<code>hotfix/<version></code>	Urgent fix to a live release	Very short	<code>main</code> and <code>develop</code>

Rules: branch names are lowercase, hyphenated, and scoped by prefix; branches are short-lived (rebase or merge `develop` frequently to avoid drift); delete branches after merge. A release branch is cut from `develop` at feature-complete; only stabilisation lands on it (Section 26). Hotfixes branch from the live tag on `main`, fix the one thing, and merge back to both `main` and `develop` so the fix is never lost.

32.4 Commit messages

Use **Conventional Commits**: a typed, imperative, present-tense subject under ~72 characters, an optional body explaining *why*, and an optional footer for issue references. Typed history makes changelogs and `git log` scannable.

Commit message format and examples

```
<type>(<scope>): <imperative summary>

[optional body: what & why, wrapped at ~72 cols]

[optional footer: Refs #123, BREAKING CHANGE: ...]

Types: feat, fix, refactor, perf, test, docs, chore, build, ci

Examples:
feat(traps): add FallingPlatformTrap with configurable delay
fix(save): fall back to fresh SaveData on corrupt file
perf(pool): reuse death VFX instances to remove GC spikes
refactor(player): move movement logic out of MonoBehaviour
test(progression): cover level-unlock boundary cases
docs(adr): record VContainer over Zenject decision
```

Commit small and often; each commit should build. Never commit broken code to `develop` or `main`. Avoid "WIP" commits on shared branches (squash them before the PR).

32.5 Pull requests

- **All work reaches `develop` / `main` via a pull request**, even solo — the PR is the review surface and the CI gate (Section 26).
- Keep PRs **small and single-purpose**; a reviewer should finish in one sitting (Section 31.12). Large PRs hide bugs and stall review.
- PR description states **what changed, why, and how it was tested**, with screenshots/GIFs for anything visual (Section 33).

- **CI must be green** (EditMode tests, build check) before merge; a red build blocks the merge (Sections 23, 26).
- At least one approving review on a team; solo, self-review against the checklist with fresh eyes the next session.
- Prefer **squash-merge** for feature branches to keep `develop` history clean (one logical change = one commit), and a regular merge for release/hotfix so the branch topology is preserved.

32.6 PR review checklist

This complements the architectural review (Section 31.12) with Git-hygiene checks:

- ☐ Branch correctly named and based on the right parent (Section 32.3)?
- ☐ Commits typed, imperative, and each one building (Section 32.4)?
- ☐ No binaries committed outside LFS; all `.meta` files present (Section 32.2)?
- ☐ No secrets, keystores, API keys, or local paths committed (Sections 26–27)?
- ☐ CI green: tests pass and the project builds?
- ☐ PR scoped small enough to review fully, with a clear description and visuals where relevant?

32.7 Tags, releases, and ignore rules

- **Tag every release on `main`** with an annotated semantic-version tag (`v1.2.0`) matching the build's version (Section 26). Tags make "what shipped" and rollbacks unambiguous.
- Keep a `CHANGELOG.md` (generated from Conventional Commits or hand-curated) so each version's changes are discoverable (Section 33).
- A correct `.gitignore` keeps Unity's generated and machine-specific folders out of the repo:

```
# Unity generated
[LL]ibrary/
[Tt]emp/
[Oo]bj/
[Bb]uild/
[Bb]uilds/
[LL]ogs/
[Uu]serSettings/
# IDE / OS
.vs/
.idea/
.DS_Store
# Build artefacts / secrets – never commit
*.apk
*.aab
*.ipa
*.keystore
*.keystore.*
secrets.*
```

Never commit signing keystores, provisioning profiles, or API keys (Sections 26–27). Store them in the CI secret manager and document the recovery process in `Documentation/` (Section 33). A leaked upload key can mean losing the ability to update your own app.

33. Documentation Strategy

Documentation is part of the product, not an afterthought. The goal is that a new contributor — or you, six months from now — can clone the repo, read a short tree of Markdown, and be productive in an afternoon. All docs live **in the repository** under `Documentation/` (Section 7) so they are versioned with the code and reviewed in the same PRs (Section 32).

33.1 Where documentation lives

- A top-level `README.md` is the front door.
- A `Documentation/` folder holds the guides, checklists, and decision records below, in Markdown so they diff cleanly and render on the Git host.
- Each guide is short and task-focused; link between them rather than duplicating. Stale duplication is worse than a single canonical page.

Documentation/ — in-repo documentation tree

```

README.md           ← project front door, quickstart
CHANGELOG.md        ← per-version changes (Section 32.7)
Documentation/
├─ Setup.md         ← clone → open → run on device
├─ Architecture.md  ← layers, assemblies, rules (Parts III–IV)
├─ DependencyInjection.md ← VContainer scopes & how to register
├─ CreatingATrap.md  ← author a new trap (Section 15)
├─ LevelDesign.md   ← author/validate a level (Section 16)
├─ Build.md         ← Android/iOS build & release (Section 26)
├─ Testing.md       ← how to run & write tests (Section 23)
├─ Checklists/
│   └─ QA.md        ← pre-release QA pass (Section 23)
│   └─ Release.md   ← store submission steps (Sections 26–27)
├─ KnownIssues.md  ← current bugs & workarounds
└─ ADR/            ← architecture decision records
    ├─ 0001-vcontainer-over-zenject.md
    ├─ 0002-clean-architecture-assemblies.md
    └─ 0003-traps-as-data.md

```

33.2 README

The README is read first and most often. Keep it to a single screen of essentials with links out to detail:

- One-paragraph **what the game is** and the working title (PLUMMET, Section 1).
- **Quickstart**: required Unity 6 version, how to clone (with LFS), open, and run on a device — pointing to `Setup.md`.
- **Repository map**: where code, content, and docs live (Section 7).
- **Key links**: architecture overview, build guide, contribution rules (coding standards + Git workflow, Sections 31–32).

33.3 Core guides

Each guide answers one question completely so contributors don't have to reverse-engineer the codebase:

- **Setup.md** — prerequisites (Unity 6 version, Git LFS), clone-to-running-on-device steps, common first-run errors. The "afternoon to productive" guarantee lives here.
- **Architecture.md** — the layer model, assembly graph, and the non-negotiable rules (Parts III–IV, Section 31.11) with the diagrams from Sections 8–9. This is the most important guide; it is why the codebase stays clean.
- **DependencyInjection.md** — how LifetimeScopes are structured (game vs level vs menu), how to register a new service, and the singleton-vs-scoped decision (Section 10).
- **CreatingATrap.md** — step-by-step: implement `ITrap` /extend `TrapBase` , author the config, place it in a level, validate fairness (Section 15). This guide is what keeps "levels are data" true in practice.
- **LevelDesign.md** — how to author a level asset, the construction and fairness rules, how to run the level validator, and the difficulty-curve guidance (Section 16).
- **Build.md** — Android and iOS build steps, signing/keystore handling, version bumping, TestFlight/Play tracks, and the CI pipeline (Section 26).
- **Testing.md** — how to run EditMode/PlayMode tests locally, what must be covered, and how CI gates merges (Section 23).

33.4 Checklists

The production checklists from this document are mirrored into the repo as living, tickable Markdown so they are used during real passes, not just read once:

- **Checklists/QA.md** — the per-release QA pass (functional, device matrix, performance against budget, fairness, store-compliance — Sections 21, 23, 27).
- **Checklists/Release.md** — the store-submission runbook (build, sign, store assets, privacy/data-safety forms, ATT, age rating, tag — Sections 26–27, and the final checklist in Section 34).

33.5 Known issues and troubleshooting

A `KnownIssues.md` lists current bugs, their severity, and any workaround, plus a short troubleshooting section for recurring environment problems (LFS not installed, wrong Unity version, missing meta files). It prevents the same questions and the same "is this a known bug?" investigations from recurring.

33.6 Architecture Decision Records (ADRs)

Significant decisions are captured as short, immutable ADRs so future contributors understand *why* the architecture is the way it is and don't relitigate settled trade-offs. One file per decision, numbered, never edited after acceptance (supersede with a new ADR instead).

ADR template (Documentation/ADR/NNNN-title.md)

```
# ADR NNNN: <short decision title>

Status: Accepted | Superseded by ADR-XXXX
Date: YYYY-MM-DD

## Context
What problem/forces led to this decision? (1–2 short paragraphs.)
```

Decision

What we chose, stated plainly.

Consequences

What this makes easy, what it makes harder, and what we accept as a trade-off. Note follow-up rules it implies.

Seed ADRs document the foundational choices: VContainer over alternatives (Section 10), the clean-architecture assembly split (Sections 8–9), and "traps/levels as data" (Sections 15–16). New ADRs are added whenever a decision is costly to reverse.

33.7 Keeping documentation alive

- **Docs change in the same PR as the code they describe.** A behaviour change with stale docs fails review (Sections 31.12, 32.6). This is the single rule that prevents rot.
- Prefer **one canonical place** for each fact and link to it; never copy-paste explanations between guides.
- Treat the README quickstart and `Setup.md` as **tested**: occasionally do a clean clone and follow them verbatim. If a step is wrong or missing, the docs are broken, not the reader.
- Keep diagrams as text/ASCII where practical (as in this document) so they live in Markdown, diff in Git, and never go missing as orphaned image files.

Part XI · Reference & Appendices

The runbook and the reference shelf. Section 34 is the single checklist you walk top-to-bottom before you ship; Section 35 collects the trees, maps, code templates, and the glossary in one place so they are easy to find without hunting through the body of the document.

34. Final Production Checklist

This is the master gate between "feature-complete" and "submitted." Walk every list before a release candidate (Section 28, RC phase). Nothing here is new policy; it is the consolidated, tickable form of the standards and budgets defined throughout this document, mirrored into `Documentation/Checklists/` (Section 33) so it is used during real passes. A box is only ticked when it is verified on a real device, not assumed.

34.1 Code

- ☐ No static managers, singletons, or service locators for game systems; everything is registered and injected (Sections 10, 31.11).
- ☐ No God classes or fat MonoBehaviours; logic lives in injected, testable services (Sections 9, 31.3).
- ☐ Feature and assembly rules hold: no feature references another feature; Core.Contracts stays engine-free (Section 8).

- ☐ EditMode tests cover progression, save migration, and trap logic, and the full suite is green in CI (Sections 23, 26).
- ☐ No per-frame allocations on hot paths; verbose logging stripped from release (Sections 21, 31.8).
- ☐ All events unsubscribed and async work cancellable; no leaks across scene reloads (Sections 12, 31.5–31.6).
- ☐ Release build compiles with IL2CPP for both platforms, no errors or stripping crashes (Section 26).

34.2 Gameplay

- ☐ All shipped levels are completable and have been beaten by a human on device (Section 16).
- ☐ Every level is "fair after discovery": no untelegraphed, unavoidable, or first-attempt-impossible deaths (Sections 2, 16).
- ☐ Restart is instant (within budget) and ad-free; the retry loop has no friction (Sections 2, 18, 25).
- ☐ Difficulty curve validated against analytics (death-to-completion and abandon rates within target) (Sections 16, 22).
- ☐ Controls feel responsive: coyote time, jump buffering, and tuning verified on device (Section 14).
- ☐ Onboarding teaches the core loop without a wall of text; first levels hook the player (Sections 16, 18).
- ☐ Save/continue works across app kills and updates; progress is never lost (Section 20).

34.3 Art

- ☐ Consistent original style across all levels; no placeholder or temp art remains (Section 19).
- ☐ Readability rules hold: hazards, interactables, and safe ground are instantly distinguishable; contrast verified (Section 19).
- ☐ Blip and all trap/hazard visuals are final and on the 16 PPU grid; no mixed pixel densities (Sections 15, 19).
- ☐ UI is legible at smallest target resolution; safe areas respected on notched devices (Section 18).
- ☐ App icon, store screenshots, and feature graphic are final and original (Sections 26–27).
- ☐ No copyrighted, trademarked, or AI-ambiguous assets; branding is original and clearable (Sections 1, 27).

34.4 Audio

- ☐ Full SFX set present (jump, land, death, trap triggers, UI, success) and mixed at consistent levels (Section 17).
- ☐ Death and retry sounds are satisfying, not punishing or grating on repeat (Sections 2, 17).
- ☐ Music (if any) loops cleanly and respects the mute/volume settings (Sections 17–18).
- ☐ Haptics fire on the right beats, are subtle, and honour the haptics toggle (Section 17).
- ☐ Audio is compressed/loaded appropriately for mobile; no oversized clips inflating the build (Sections 17, 21).
- ☐ Sound and haptics settings persist via the save system (Sections 18, 20).

34.5 Mobile performance

- ☐ Sustained target frame rate on a mid device; 30 FPS floor verified on a genuine low-end Android (Sections 4, 21).
- ☐ Restart, level-load, and memory all within the performance budget on device (Sections 4, 21).
- ☐ No GC spikes during gameplay or restart; pooling in place for VFX/effects (Sections 21, 31.11).
- ☐ Draw calls within budget; sprite atlasing and batching verified (Sections 4, 21).
- ☐ Build size within target for both stores; assets stripped of unused content (Sections 4, 26).
- ☐ Battery and thermals sane over a long session; no runaway frame rate or background work (Section 21).
- ☐ Profiled on device (not just editor) at the RC stage (Sections 21, 23).

34.6 Monetization

- ☐ Game is fully completable without spending or watching a single ad (Section 25).
- ☐ No ads after death; the retry loop stays instant and ad-free (Sections 18, 25).
- ☐ No interstitials during onboarding/first levels; frequency caps in place and tuned (Section 25).
- ☐ Rewarded ads are strictly optional and never gate progress (Section 25).
- ☐ Ads/IAP behind interfaces (swappable adapters), with graceful failure that never blocks play (Sections 8, 25, 31.7).
- ☐ Store listing and pricing accurate; restore-purchases works where applicable (Sections 25–26).

34.7 Release

- ☐ Version and build numbers bumped and tagged on `main` (Sections 26, 32.7).
- ☐ Signed builds produced for both platforms (keystore/provisioning handled via CI secrets) (Section 26).
- ☐ Privacy policy published; data-safety/privacy-nutrition forms completed accurately (Section 27).
- ☐ iOS ATT prompt (if tracking) and consent defaults (off) implemented and tested (Section 27).
- ☐ Honest age rating completed for both stores (Section 27).
- ☐ Store assets final: icon, screenshots, description, keywords, feature graphic (Sections 26–27).
- ☐ Crash reporting and analytics verified live in the release build (Sections 22, 26).
- ☐ Internal/TestFlight + Play closed track passed before production rollout; staged rollout planned (Section 26).
- ☐ Rollback plan and hotfix branch process understood before submitting (Sections 26, 32.3).

35. Appendices

These appendices consolidate the reference material referenced throughout the document. Where an appendix summarises a topic covered in depth earlier, it points to the authoritative section rather than duplicating it; the code templates (Appendices E–F) are concrete starting points to copy from.

Appendix A · Full Folder and File Tree

The authoritative, fully annotated tree is in Section 7. The condensed top-level view below is the quick map; everything under `Code/` is organised feature-first (Sections 8–9): a shared `Core`, one folder per feature, and a single `App` composition root.

Assets/_Project — condensed top-level map (full tree in Section 7)

```
Assets/_Project/
├─ Art/           sprites, animations, atlases, fonts (LFS)
├─ Audio/         sfx, music (LFS)
├─ Code/
│   ├─ Core/
│   │   ├─ Contracts/ interfaces, signals, shared data    (Game.Core.Contracts)
│   │   └─ Runtime/   shared services + DI scaffolding    (Game.Core)
│   └─ Features/
│       ├─ Player/    movement, view, config, installer  (Game.Features.Player)
│       ├─ Traps/      trap framework + configs           (Game.Features.Traps)
│       ├─ Levels/     level loading + LevelData          (Game.Features.Levels)
│       ├─ Progression/ unlock rules (engine-free)        (Game.Features.Progression)
│       └─ Ui/         presenters + views                 (Game.Features.Ui)
│   └─ App/           LifetimeScopes + entry points       (Game.App)
│   └─ Editor/        editor tooling                     (Game.Editor)
│   └─ Tests/PlayMode/ integration tests                 (Game.Tests.PlayMode)
├─ Content/         Levels/*.asset, Configs/*.asset (designer data)
├─ Prefabs/         player, traps, UI prefabs
├─ Scenes/          Boot, MainMenu, Loading, Game
├─ Settings/        URP, input, build, quality
└─ Documentation/   guides, checklists, ADRs (Section 33)
```

Each Features/<X>/ slice carries its own Runtime/, Config/, DI/, and Tests/.

Appendix B · Assembly Dependency Map

The assemblies and their rules are detailed in Section 8. The dependency direction is the architecture's backbone: every feature points at the shared `Core.Contracts`, no feature points at another feature, and `App` sits on top referencing everything. Unity forbids cycles, so the star-shaped graph cannot rot.

Assembly dependencies (full table & rules in Section 8)

```
Game.Core.Contracts    → (nothing; engine-light API surface)
Game.Core              → Core.Contracts
Game.Features.Player   → Core.Contracts, Core
Game.Features.Traps    → Core.Contracts, Core
Game.Features.Levels   → Core.Contracts, Core
Game.Features.Ui       → Core.Contracts, Core
Game.Features.Progression → Core.Contracts          (engine-free)
Game.App               → Core.Contracts, Core, ALL Game.Features.*
Game.Editor            → (selected runtime assemblies), UnityEditor
Game.Features.<X>.Tests → Game.Features.<X>, Core.Contracts
Game.Tests.PlayMode    → Game.App + exercised features
```

Rules: feature → Core only (never feature → feature); Core → Core.Contracts only (never Core → feature); App → everything (the single composition root). Cross-feature collaboration goes through Core.Contracts + the SignalBus (Sections 9, 31.11).

Appendix C · VContainer Lifetime-Scope Reference

Full scope classes are in Section 10 (`GameLifetimeScope`, `LevelLifetimeScope`) and each feature's installer (`PlayerInstaller`, ...). The cheat-sheet below is the registration vocabulary to reach for; remember that **features register their own services in their own installer**, and the App scopes just install features and register Core services (Sections 9–10).

Assets/_Project/Code/App/DI/RegistrationCheatSheet.cs · asmdef: `Game.App`

```
// Inside Configure(IContainerBuilder builder) in an App LifetimeScope:

// Core app-wide singleton (one instance for the whole app): save, audio, analytics
builder.Register<ISaveService, JsonSaveService>(Lifetime.Singleton);
builder.Register<ISignalBus, SignalBus>(Lifetime.Singleton);

// Install a feature – the feature owns its own registrations
new PlayerInstaller(playerConfig, playerView).Install(builder);
new TrapsInstaller().Install(builder);

// (Inside a feature installer:) register that feature's own service, Scoped
builder.Register<PlayerMovementService>(Lifetime.Scoped);

// Existing scene component (MonoBehaviour placed in the level prefab)
builder.RegisterComponentInHierarchy<PlayerView>();

// New component on a fresh GameObject, created & injected by the container
builder.RegisterComponentOnNewGameObject<HudView>(Lifetime.Scoped);

// Entry point: runs Start/Tick via the container (no MonoBehaviour)
builder.RegisterEntryPoint<LevelEntryPoint>();

// Bind a value/config object (e.g. a loaded ScriptableObject)
builder.RegisterInstance(playerMovementConfig);
```

Rule of thumb: **Singleton** for Core's cross-cutting services that hold app-lifetime state (save, audio, analytics, signal bus); **Scoped** for a feature's per-level services so they are cleanly recreated on reload (Sections 10, 12). A feature never `new`s a sibling feature's service — it asks for a `Core.Contracts` interface and the container supplies whatever was installed (Section 31.11).

Appendix D · Service Interface Reference

The shared contracts the whole game depends on, all defined in `Game.Core.Contracts` (engine-light) and implemented by concrete services in `Game.Core` (Sections 8–9). Features depend on these interfaces, never on each other's concretes. Full definitions appear in the referenced sections.

Interface	Defined in	Implemented in	Responsibility	Section
<code>IGameStateService</code>	<code>Core.Contracts</code>	Core	Owns the current <code>GameState</code> and transitions; raises state-changed signals	13

Interface	Defined in	Implemented in	Responsibility	Section
<code>IPlayerInput</code>	<code>Core.Contracts</code>	<code>Core</code>	Abstract input (move/jump intent) so features never touch <code>UnityEngine.Input</code>	14
<code>ITrap</code>	<code>Core.Contracts</code>	<code>Features.Traps</code>	The trap contract: arm / trigger / activate / reset lifecycle	15
<code>ISaveService</code>	<code>Core.Contracts</code>	<code>Core</code>	Load/save/delete persisted <code>SaveData</code> ; hides storage backend	20
<code>IObjectPool<T></code>	<code>Core.Contracts</code>	<code>Core</code>	Generic pooling contract for reusable instances	21
<code>IAalyticsService</code>	<code>Core.Contracts</code>	<code>Core</code>	Fire-and-forget telemetry; never blocks gameplay	22
<code>IAdsService</code>	<code>Core.Contracts</code>	<code>Core</code>	Optional rewarded/interstitial ads; graceful failure, swappable vendor	25
<code>IAudioService</code>	<code>Core.Contracts</code>	<code>Core</code>	Play SFX/music; honours the sound settings	17
<code>ISceneLoader</code>	<code>Core.Contracts</code>	<code>Core</code>	Scene/Addressable loading behind an interface	12
<code>ISignalBus</code>	<code>Core.Contracts</code>	<code>Core</code>	Publish/subscribe for cross-feature signals (e.g. <code>PlayerDiedSignal</code>)	9

These interfaces are the seam that keeps features independent and vendors swappable. `ITrap` is the one contract *implemented by a feature* (`Features.Traps`) rather than by `Core` — it lives in `Core.Contracts` so the `Levels` feature can place and reset traps without ever referencing the `Traps` feature (Sections 9, 15). A new backend (cloud save, a different ads SDK) is a new `Game.Core` implementation of the same interface — no change to any feature (Sections 8, 25, 31.11).

Appendix E · Trap Configuration Example

A concrete companion to the trap code in Section 15. Trap *behaviour* is code in the `Traps` feature (`ITrap / TrapBase`); trap *tuning* is a designer-authored `ScriptableObject` that derives from the shared `TrapConfig` base in `Core.Contracts` , so the `Levels` feature can place and reference any trap by its base type without depending on the `Traps` feature (Sections 9, 15–16, 31.4).

Assets/_Project/Code/Features/Traps/Config/FallingPlatformTrapConfig.cs · asmdef: `Game.Features.Traps`

```
using UnityEngine;
using Game.Core.Contracts.Traps; // the shared TrapConfig base

namespace Game.Features.Traps.Config
{
    [CreateAssetMenu(
        fileName = "FallingPlatformTrapConfig",
        menuName = "Plummet/Traps/Falling Platform")]
}
```

```

public sealed class FallingPlatformTrapConfig : TrapConfig
{
    [Tooltip("Seconds after the player steps on it before it falls.")]
    [Min(0f)] public float TriggerDelay = 0.25f;

    [Tooltip("Downward acceleration once it lets go.")]
    [Min(0f)] public float FallAcceleration = 30f;

    [Tooltip("Seconds before the platform resets to its start position.")]
    [Min(0f)] public float ResetDelay = 1.5f;

    [Tooltip("Does it shake to telegraph the fall? (fairness, Section 16)")]
    public bool TelegraphShake = true;

    void OnValidate()
    {
        // Catch unfair/zero values at author time, not on a device.
        if (TelegraphShake && TriggerDelay < 0.1f)
            Debug.LogWarning(
                $"{name}: TriggerDelay is very low for a telegraphed trap; " +
                "the player may not get a fair warning (Section 16).");
    }
}

```

The trap `MonoBehaviour / TrapBase` (Section 15) in `Features.Traps` holds a reference to this config and reads it at runtime; it never hardcodes these numbers. Designers tune difficulty by editing assets, and `OnValidate` guards fairness before the value ever reaches a player.

Appendix F · Level Data Example

A concrete companion to the level pipeline in Section 16. A level is a data asset describing what to place and where — never a bespoke script (Sections 15–16, 31.11). The `Levels` feature reads it; designers author it. Trap placements reference the shared `TrapConfig` base (Core.Contracts, Appendix E), so `Levels` stays independent of the `Traps` feature.

Assets/_Project/Code/Features/Levels/Data/LevelData.cs · asmdef: `Game.Features.Levels`

```

using System;
using System.Collections.Generic;
using UnityEngine;
using Game.Core.Contracts.Traps; // shared TrapConfig base – not the Traps feature

namespace Game.Features.Levels.Data
{
    [Serializable]
    public struct TrapPlacement
    {
        public TrapConfig Config; // a config deriving from TrapConfig
        public Vector2 Position;
        public float RotationDegrees;
    }

    [CreateAssetMenu(
        fileName = "Level_00",

```

```

    menuName = "Plummet/Level"]
    public sealed class LevelData : ScriptableObject
    {
        [Min(1)] public int LevelNumber = 1;

        public Vector2 SpawnPoint;
        public Vector2 ExitPoint;

        [Tooltip("Designer 'par' time in seconds, for analytics (Section 22).")]
        [Min(0f)] public float ParTimeSeconds = 8f;

        public List<TrapPlacement> Traps = new();

        void OnValidate()
        {
            if (ExitPoint == SpawnPoint)
                Debug.LogWarning($"{name}: spawn and exit are identical.");
        }
    }
}

```

The level loader (Section 16) instantiates the player at `SpawnPoint`, the goal at `ExitPoint`, and each trap from its placement + config. Because levels are assets and reference only the shared `TrapConfig` base, authoring level 51 costs no code and no recompile, the `Levels` feature never depends on the `Traps` feature, and the level validator (Sections 16, 31.10) can check every asset in CI.

Appendix G · Mobile Optimization Checklist

The reference form of the techniques in Section 21, applied continuously during development (not just at the end). Verified on a genuine low-end device, against the budget in Sections 4 and 21.

- ☐ Sprites packed into atlases; draw calls and SetPass calls within budget (Sections 4, 21).
- ☐ URP configured for mobile (no unnecessary post-processing, HDR, or MSAA cost) (Sections 6, 21).
- ☐ Object pooling for all frequently spawned objects (death VFX, particles); zero per-restart GC spikes (Section 21).
- ☐ No per-frame allocations: cache references, avoid LINQ/boxing/string concat in `Update` (Sections 21, 31.8).
- ☐ Physics tuned: minimal fixed timestep work, simple colliders, no idle physics churn (Section 21).
- ☐ Audio clips compressed and loaded appropriately (streamed vs decompressed) for size and memory (Sections 17, 21).
- ☐ Textures sized and compressed per platform (ASTC), mipmaps off for UI/2D where unneeded (Section 21).
- ☐ Memory under budget with no leaks across level reloads (events unsubscribed, instances pooled) (Sections 12, 21).
- ☐ Stable target frame rate on mid devices; 30 FPS floor verified on low-end Android (Sections 4, 21).
- ☐ Build size stripped of unused assets/code; within store target (Sections 4, 26).

- ☐ Profiled on device with the Profiler/Frame Debugger at least once per phase (Sections 21, 23).

Appendix H · QA Checklist

The reference form of the testing strategy in Section 23, run as a pass before every release candidate (Section 28). Combine automated suites (run in CI, Section 26) with the manual device pass below.

- ☐ EditMode and PlayMode suites pass in CI; coverage includes progression, save migration, and trap logic (Sections 23, 26).
- ☐ Every shipped level completed by a human on device; no soft-locks or impossible states (Section 16).
- ☐ Fairness pass: no untelegraphed/unavoidable deaths; "fair after discovery" holds (Sections 2, 16).
- ☐ Save/continue verified across app kill, backgrounding, and version upgrade (Section 20).
- ☐ Corrupt/missing save falls back gracefully to a fresh state (Sections 20, 31.7).
- ☐ Settings (sound, haptics) persist and take effect immediately (Sections 18, 20).
- ☐ Device matrix covered: smallest/notched screens, low-end Android, current iOS (Sections 4, 23).
- ☐ Performance within budget on device (frame rate, restart, load, memory) (Sections 4, 21).
- ☐ Ads/IAP paths tested including failure: gameplay never blocked, restore works (Sections 25, 31.7).
- ☐ Analytics events fire correctly in a release build; no debug spam (Sections 22, 31.8).
- ☐ Interruptions handled: phone call, notification, low battery, app switch mid-level (Section 23).
- ☐ Localisation/text fits and is legible at the smallest target resolution (Section 18).

Appendix I · Release Checklist

The reference form of the build/deploy and compliance runbook (Sections 26–27), consolidating the release portion of Section 34 for use at submission time.

- ☐ Version and build numbers bumped; release tagged on `main` (Sections 26, 32.7).
- ☐ Signed Android (AAB) and iOS (IPA) builds produced via CI; keystore/provisioning from secrets, never committed (Sections 26, 32.7).
- ☐ IL2CPP release builds verified on device; no stripping/crash regressions (Section 26).
- ☐ Privacy policy published and linked in both stores (Section 27).
- ☐ Google Play Data Safety and Apple Privacy "nutrition labels" completed accurately (Section 27).
- ☐ iOS ATT prompt (if tracking) implemented; consent defaults off and respected (Section 27).
- ☐ Honest content age rating completed for both stores (Section 27).
- ☐ Store listing final: title, subtitle, description, keywords, original icon, screenshots, feature graphic (Sections 26–27).
- ☐ Crash reporting and analytics confirmed live in the production build (Sections 22, 26).
- ☐ Internal/TestFlight and Play closed-track validation passed before production (Section 26).
- ☐ Staged/phased rollout configured; monitoring dashboards ready (Sections 22, 26).
- ☐ Rollback and hotfix process rehearsed; `hotfix/*` branch flow understood (Sections 26, 32.3).

- ☐ Trademark/store-name availability confirmed for the final title (Sections 1, 27).

Appendix J · Glossary

Adapter

A concrete service in `Game.Core` that connects a shared contract to a specific technology (file system, ads SDK, Unity input). Adapters implement `Core.Contracts` interfaces and are wired in at the composition root (Sections 8–9).

ADR (Architecture Decision Record)

A short, dated, immutable note capturing a significant decision and its trade-offs, stored in `Documentation/ADR/` (Section 33.6).

App (composition root)

`Game.App` — the only assembly that references every feature. Its `LifetimeScope`s register Core's services and install each feature's installer, assembling the slices into a running game (Sections 9–10).

asmdef (Assembly Definition)

A Unity file that compiles a folder into its own assembly with explicit references, enforcing feature isolation and speeding incremental compilation (Section 8).

ATT (App Tracking Transparency)

Apple's required prompt before tracking a user across apps for advertising; gated and off by default in this project (Section 27).

Atlas (sprite atlas)

A packed sheet combining many sprites so they render in fewer draw calls — a core 2D mobile optimisation (Sections 19, 21).

Blip

The game's original hero — a faceless matte-black silhouette block. The player character of PLUMMET (Sections 1, 19).

Composition root

The single place where Core services are registered and features are installed. Here, the `LifetimeScope`s in `Game.App` — nowhere else (Sections 10, 31.11).

Contract

A shared interface (or signal, or data type) in `Game.Core.Contracts` that features depend on, implemented by a concrete service (usually in `Game.Core`, or in a feature for `ITrap`). The seam that keeps features independent and technology swappable (Sections 8–9).

Conventional Commits

A commit-message convention (`type(scope): summary`) that yields scannable history and easy changelogs (Section 32.4).

Core

The shared foundation every feature builds on, in two assemblies: `Game.Core.Contracts` (interfaces, signals, shared data) and `Game.Core` (concrete shared services + DI scaffolding). Core never depends on a feature (Sections 8–9).

Coyote time

A short grace window after leaving a ledge during which a jump still registers, making controls feel fair (Section 14).

Dependency Injection (DI)

Supplying a class's dependencies from outside (via constructor/scope) rather than having it create or locate them, enabling testability and decoupling (Section 10).

Draw call

A command to the GPU to render something; minimising draw calls (via atlasing/batching) is key to mobile performance (Sections 4, 21).

EntryPoint

A VContainer-managed object that receives lifecycle callbacks (`Start` / `Tick` / `FixedTick`) without being a `MonoBehaviour`, keeping logic out of the scene graph (Section 10).

Feature (slice)

A self-contained vertical slice of the game (Player, Traps, Levels, Progression, Ui), one assembly, owning its runtime, views, config, DI installer, and tests. A feature depends only on Core and never on another feature (Section 9).

Feature installer

A small class (implementing `IFeatureInstaller`) that registers a feature's own services with the DI container. The App's scopes install features by calling their installers, so each feature owns its wiring (Section 10).

Feature-first architecture

Organising code by vertical slice (Player, Traps, ...) rather than by technical layer. Slices depend only on a shared Core and never on each other; the App composes them. Also called vertical-slice or feature-folder architecture (Section 9).

GC spike

A frame hitch caused by garbage collection of runtime allocations; avoided by pooling and zero-allocation hot paths (Section 21).

Git LFS (Large File Storage)

A Git extension that stores large binaries (art, audio) outside normal history to keep the repo lean (Section 32.2).

IL2CPP

Unity's ahead-of-time backend that converts IL to C++ for performant, store-compliant native builds on iOS/Android (Section 26).

Jump buffering

Registering a jump pressed slightly before landing so it fires on touchdown, making input feel responsive (Section 14).

LifetimeScope

A VContainer container defining what is registered and how long instances live (e.g. a root scope and a per-level scope). All scopes live in `Game.App` (Section 10).

MonoBehaviour

A Unity component attached to a `GameObject`. In this project it is a thin adapter (input/lifecycle/rendering), never a home for game logic (Section 31.3).

Object pooling

Reusing a fixed set of pre-allocated instances instead of instantiating/destroying at runtime, eliminating GC spikes (Section 21).

PLUMMET

The working title of the original troll platformer described by this document; tagline "It looks easy" (Section 1).

PPU (Pixels Per Unit)

How many sprite pixels map to one Unity world unit; a consistent value (16 here) keeps pixel art crisp and uniform (Section 19).

Scoped (lifetime)

A DI lifetime giving one instance per LifetimeScope — used for a feature's per-level services so they are recreated cleanly on reload (Sections 10, 12).

ScriptableObject

A Unity data asset used here for designer-authored, immutable configuration and content (levels, trap tuning) — never as mutable global state (Section 31.4).

Signal / SignalBus

A typed publish/subscribe message defined in `Game.Core.Contracts` and dispatched by the `ISignalBus` (in `Game.Core`). The decoupled way features collaborate without referencing each other — e.g. Player publishes `PlayerDiedSignal`; UI and Progression subscribe (Section 9).

Singleton (lifetime)

A DI lifetime giving one instance for the whole app — used for Core's cross-cutting services (save, analytics, audio, signal bus). Distinct from the banned static-singleton *pattern* (Sections 10, 31.11).

Trap

In PLUMMET, an original hazard with an arm → trigger → activate → reset lifecycle, configured by data and placed in levels. Trap behaviour lives in the `Traps` feature; the `ITrap / TrapConfig` contracts live in Core (Section 15).

URP (Universal Render Pipeline)

Unity's scalable render pipeline, configured for cheap, consistent 2D rendering on mobile here (Sections 6, 21).

VContainer

The lightweight, fast dependency-injection framework used for all composition in this project (Section 10).